Seat Ase?

System Design Document

A Dhaka Tesla-Pool Ride-Sharing MVP

Find a seat. Find a ride. Stay strangers.

Document	
Author	Shajid Shahriar
Prepared for	RoBenDevs — SWE Intern Take-Home Assessment
Version	6.0
Stack	Next.js (frontend) · Express (backend) · PostgreSQL (database) · Docker
Scope	Product decisions, system architecture, data model, concurrency control, API design, security, testing strategy, deployment, scalability plan, and a pre-implementation risk analysis.

How this document is organised

Sections follow a consistent structure for every non-trivial decision: what was chosen, what the alternatives were, why the choice fits this product, and under what conditions it would change. Diagrams and ERDs are drawn directly from the implementation and are reusable in the project README.

1. Product Overview

Seat Ase? is a scaled-down Uber-Pool for Dhaka: passengers request a ride, a driver accepts them, and several passengers travelling in roughly the same direction can share one vehicle and each pay less. The vehicle in question is a 'Tesla' — Dhaka slang for a three-wheeled battery rickshaw, named for its daring attitude on the road, not the car company.

A fixed cast of personas (Jashim the driver, and passengers Nusrat, Rafiq and Shirin) is used consistently across the seed data, automated tests, documentation and demo video, so that every fare and pooling decision in this document is a concrete, independently verifiable number rather than an abstract example.

The three actors

Who	What they can do
Passenger Nusrat, Rafiq, Shirin	Sign up, verify phone, log in. Choose pickup and drop-off (search or map pin) and seats. Pick **Share** or **Private**; women can ask for **women-only**. See a fare estimate. Watch the ride move through its stages. See who they're sharing with. See past rides. Cancel while it's still allowed.
Driver + car Jashim and Bullet (3 seats)	Sign up, log in, **add his vehicle** (name, registration number, seats). Go online / offline. See ride requests. Accept them. Press 'arrived', 'start', then **drop off** each passenger (or mark a no-show). See who is in the car and past rides.
The pool (a shared ride)	Many requests can sit in one car. The car can **never** hold more people than seats. Every passenger gets their own fare. It's always obvious who is in which pool.

Test Personas and Scenarios

The same named personas are used throughout — seed data, automated tests, README and demo — rather than generic placeholders, so that every example in this document (and every assertion in the test suite) refers to the same, checkable trip.

Cast	Role	Details (fixed everywhere)
Jashim	Driver	Drives **Bullet**, 3 seats.
Mokbul	Driver (our addition)	Drives **Toofan**, 3 seats. Exists for the two-driver race and the women-only ride.
Nusrat	Passenger, female	Banani → Mohakhali, 1 seat.
Rafiq	Passenger, male	Banani → Gulshan 1, 1 seat (2 seats in Scenario B: travelling with a colleague).
Shirin	Passenger, female	Banani → Mohakhali, 1 seat, usually asks for women-only.

We use five fixed **scenarios**. Each one appears the same way in tests, README and video. They are **scripts**, not seed data: the seed can't hold them all at once, because Bullet can only have one active ride and each passenger one active booking. A demo-only endpoint (`POST /dev/scenario/A`) resets and sets up any scenario, and the automated test suite runs the same scripts:

Scenario	What happens	Proves
A. Morning pool	Jashim accepts Nusrat, then Rafiq (fits). Shirin's women-only request doesn't fit Bullet because Rafiq is on board.	Pooling rule, fares 96.25 / 77.50
B. Last-seat race	Bullet carries Rafiq + colleague (2 seats). Nusrat and Shirin (1 seat each, sharing allowed) race for the last seat.	Concurrency
C. Women-only	Mokbul accepts Shirin's women-only request into Toofan. Toofan now carries a women-only booking, so male requests show 'women-only ride'.	Safety rule R5
D. Private	Nusrat hires the whole of Bullet privately: Tk 345.00 (all 3 seats), picked up at her door. Nobody else can join.	Private hire, rule R6
E. Three stops	Nusrat (Mohakhali), Rafiq (Gulshan 1) and Shirin (Tejgaon, not women-only this trip) share Bullet from the Banani Road 11 stand. Jashim drops them in the suggested order and collects Tk 288.75.	Drop-off order, per-passenger drop, fares

Cast and scenario definitions live in a single fixture file consumed by the seed script, the test suite and the README credentials table, so the three can never drift out of sync. Preferences such as women-only are per-trip rather than per-person, so Shirin choosing differently across scenarios is expected, not a contradiction.

Assessment Requirements Summary

Area	Requirement
Code	React/Next.js frontend, Node.js backend, a relational database selected and justified by the implementer.
Docker	`docker compose up` must start everything: app, database, migrations (table setup), seed data (Jashim, Nusrat, Rafiq…).
Deploy	Free hosting only. Never pay. Public link preferred.
Docs	README with architecture diagram, database diagram (ERD), setup steps, env variables, demo logins, API list, trade-offs, limitations, AI usage section, video link.
Tests	Capacity never exceeded. Bad status jumps rejected. Nusrat's and Rafiq's pooled fares correct. Users can't touch other users' rides. Cancel rules hold. Two requests at the same instant can't break seat counts.
Git	Branches: master, pre-release, release/v1.0.0, and feature/* for each piece of work. Commit messages like `feat(pool): enforce Bullet's seat capacity`. No giant single commit.
Video	Max 6 minutes: 1 min problem, 2 min engineering, 3 min product tour.

Central Engineering Challenge: The Last-Seat Race

> **Concurrency**
>
> Bullet has one seat remaining. Two passengers submit a request for it at effectively the same instant, and both clients have already been told 'one seat free'. The system must guarantee that exactly one of them succeeds and the vehicle never exceeds capacity. Section 9 covers this in full; it is the design decision this document leans on most heavily.

Design Priorities

The assessment weights engineering process — data integrity, testing, documentation, and git discipline — above raw feature count. The design below optimises for four things over breadth:

- **Bulletproof core.** Seats, statuses, fares and permissions are enforced in the database, not just in the UI.
- **Demonstrable correctness.** A live 'Seat Race' tool exercises the concurrency guarantee under real, observable contention.
- **Defensible choices.** Every non-mandated tool has a documented rationale and a stated condition for switching away from it.
- **Traceable history.** Git history structured as an incremental engineering record, not a single commit.

2. Product Decisions

The following decisions resolve ambiguity intentionally left open by the assignment brief. Each is implemented consistently throughout the system and is treated as a documented assumption (Section 18).

Question	Decision	Why
Who decides who rides?	**Jashim picks.** He sees waiting requests, each labelled 'fits' or 'doesn't fit' with a reason. The server re-checks when he taps accept.	Driver stays in control; no auto-matching engine to build; rules still enforced by the server.
How do people choose places?	**Uber-style:** search our own list of Dhaka places or drop a pin on a free map. Every point snaps to a zone.	Feels real, costs nothing, fares stay hand-checkable. Section 5.
Where are shared rides picked up?	At the nearest real **Tesla stand** (1–3 per zone), shown with walk distance. Private rides pick up at the door.	That's how shared Teslas already operate in Dhaka: passengers walk to a stand and wait for seats to fill. The app removes the uncertainty in that process.
How does the app know where Teslas are?	It doesn't track GPS. Jashim **declares his area** when going online; after his last drop-off it becomes where he ended up. Passengers see '1 Tesla online near Banani'.	Honest simulation, stated in the README; real tracking is part of the scaling story.
How do screens update?	**Instantly, with SSE** (not webhooks). Polling as backup.	Explained below.
Driver signup	Account → login → **add vehicle** (name, registration number unique, seats 1–6). Can't go online without a vehicle.	No document upload or admin approval in MVP; README says production would verify vehicles first.
Phone verification	**Simulated OTP.** Real table, hashed codes, 5-minute expiry, 5 attempts. In demo mode the code shows on screen instead of SMS. Seeded cast is pre-verified.	Real SMS costs money and the assignment brief forbids paying. We design the real thing and only fake delivery.
Identity (NID)	**Simulated NID verification** at signup: format check (10, 13 or 17 digits), marked verified instantly in demo mode. We store only a **hash** (one NID = one account) and the **last 4 digits**. **Gender is set at signup and can never be changed.**	Backs the women-only feature: nobody can make a second account or flip their gender to get into a women-only ride. Never storing the full NID is the privacy point.
Share or hire the whole Tesla?	Both. **Private** = whole Tesla, nobody joins, fare = seat fare x the Tesla's seat count (the full vehicle is paid for, seat by seat).	The assignment brief says share 'when it makes sense', so sharing is optional. Section 6.
Safety	**Women-only** option for female passengers (rule R5). Only passengers turn it on; the driver has no women-only switch. Passengers see co-riders' **first name + 'phone verified'** badge, never phone, fare or exact drop point.	Real Dhaka problem the assignment brief ignored. Keeps the assignment's rule that each passenger sees only their own fare.
8 people request, Bullet has 3 seats	Nobody is auto-booked. All 8 wait in the **Banani waiting list**; Jashim (and Mokbul) accept from it; the other 5 keep waiting for the next Tesla.	Nobody is in the car until accepted, so there is nothing to 'kick out'. Section 5.

Question	Decision	Why
Can Jashim kick someone out?	**No.** Before accepting he can skip anyone. After accepting he can only cancel the **whole** ride (everyone goes back to waiting) before arriving.	Stops drivers removing people for their looks or gender after they've stopped looking for other rides.
Passenger doesn't show up	After arriving, Jashim can mark one passenger **No-show**, but only after waiting **5 minutes**. They're removed from the trip and don't count for the pool discount.	The one legit reason to remove someone.
Getting off at different places	**Per-passenger drop-off** in a **suggested order** (nearest first). Jashim taps 'Drop Nusrat' at Mohakhali; Rafiq rides on. He sees each passenger's cash fare. The ride completes when the last passenger is dropped.	That's how pooling really works.
Refill a seat mid-trip? Pick up at several spots?	**No.** Both need to know where Jashim is while driving.	No GPS or routing in scope. Listed as next improvements.
Cancel mid-ride	Not in the app after start. The passenger tells the driver; he drops them early. **Fare doesn't change** (it was locked at start). Driver can't cancel mid-ride in MVP (breakdown = known limitation).	Simple, fair, and honest about what's out of scope.

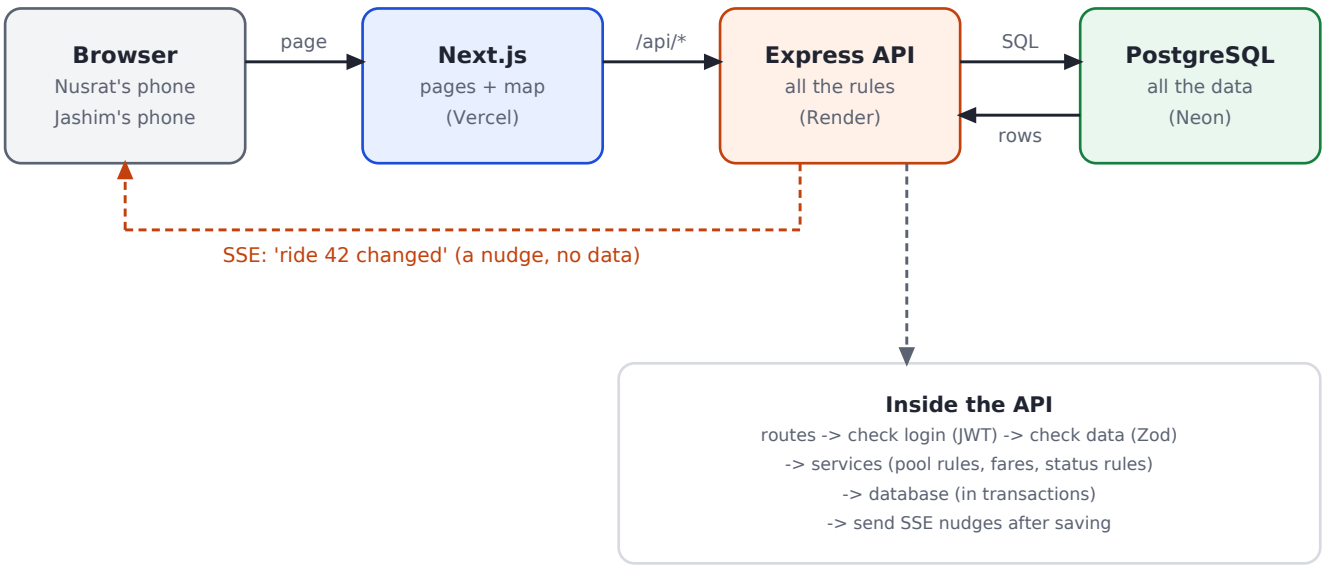
Webhook vs WebSocket vs SSE (the simple version)

Thing	Real-life version	Good for us?
Webhook	One company's server phoning another company's server. Example: bKash's server notifies a merchant's server that a payment completed. A browser has no equivalent address, so it cannot receive this kind of call.	No. It's server-to-server.
WebSocket	A phone call. Both sides can talk any time.	Works, but we don't need the browser to talk back this way. More moving parts.
SSE	A radio station. The server talks, the browser listens. When the browser wants to say something, it uses a normal request.	Yes. Status updates only go server to browser. Built into every browser, auto-reconnects, and it's just a normal HTTP response that never ends.
Polling	Asking 'are we there yet?' every 5 seconds.	Our backup plan if SSE ever fails on the hosting.

> **Note**
>
> Our trick: the SSE message is just a **nudge**, like 'ride 42 changed'. It never contains the data itself. When a screen gets the nudge, it re-asks the server for **its own** view through the normal API. That way the privacy rules (Rafiq can't see Nusrat's fare) live in one place only, the API.

3. System Architecture

The system is intentionally four components. No message queue, cache layer or microservice split is introduced at this scale — added complexity is justified only when a growth-stage metric calls for it (Section 16).



What each box does

Box	Job	What it must NOT do
Browser	Shows screens, sends clicks.	Be trusted. Anything from the browser could be faked.
Next.js	Pages, forms, map, loading/error states. Forwards `/api/*` calls to Express so cookies stay simple.	Hold business rules. It only displays.
Express API	Login, validation, ride rules, seat counting, fares, status changes, permissions, live nudges.	Trust the frontend's math. It recalculates everything.
PostgreSQL	Stores everything. Enforces the last line of defence: seat limits, unique rules, valid links.	Nothing, it's the boss of truth.

The browser never holds business logic; Next.js serves and proxies but performs no ride-domain computation; Express owns every rule and re-derives every value it is given; PostgreSQL is the single source of truth, enforced by constraints that hold even if application code has a bug.

Why the Next.js box forwards calls instead of the browser calling Express directly

When the browser talks to two different origins (the Vercel site and the Render API), login cookies become unreliable: browsers treat cross-site cookies with suspicion and some block them. So Next.js is set up to forward any `/api/...` request to Express (this is called a **rewrite** or proxy). To the browser, everything looks like one website, so the login cookie just works. Locally in Docker it's the same trick.

4. Technology Stack and Rationale

For every tool not mandated by the assignment brief, this section states what was chosen, the realistic alternatives, why the choice fits a ride-pooling MVP specifically, and the condition under which the choice would change.

Decision: Language	
We picked	**JavaScript** (plain, no TypeScript), with **Zod** for runtime validation and **JSDoc** for editor support.
Other options	TypeScript.
Why this fits	The failure modes that matter here -- overbooking, illegal status transitions, cross-user data access -- occur with real data while the application is running. TypeScript checks are compile-time only and provide no runtime guarantee. Zod validates every incoming request, and database constraints validate every write, which is where these failure modes are actually caught. JSDoc restores editor autocomplete without TypeScript syntax, and Zod schemas are shared between frontend and backend so there is a single definition of a valid ride request.
When we'd switch	A larger team, a codebase with a multi-year lifespan, or extensive refactoring -- conditions under which static typing's maintenance value outweighs its runtime cost.

Decision: Frontend framework	
We picked	**Next.js** (App Router) with Tailwind CSS.
Other options	Plain React + Vite + React Router.
Why this fits	Recommended by the assignment brief. File-based routing keeps the mapping from URL to page explicit. Built-in rewrites forward `/api` calls to Express (Section 3), and Vercel offers first-class, free hosting for the framework.
When we'd switch	If server rendering and rewrites were not required, Vite would be a lighter alternative.

Decision: Backend framework	
We picked	**Express** (Node.js).
Other options	NestJS (structured, heavier), Fastify (faster, built-in schema validation), or API routes embedded in Next.js.
Why this fits	The database, not the application server, is the throughput bottleneck at this scale, so Express's minimalism is not a liability. A separate Express app enforces a clean boundary between presentation (Next.js) and business rules (Express), and Server-Sent Events are straightforward to implement by holding a response open.
When we'd switch	A larger team benefiting from enforced structure (NestJS), or a workload where Fastify's request-handling speed matters.

Decision: Database	
We picked	**PostgreSQL 16**.
Other options	MongoDB, MySQL, SQLite.

Decision: Database	
Why this fits	The domain is fundamentally relational (a booking belongs to a ride; a ride belongs to a vehicle; a vehicle belongs to a driver) and centres on safely counting seats under concurrent access. PostgreSQL provides foreign keys, CHECK constraints (`seats_taken <= capacity`), partial unique indexes ('one active ride per vehicle'), and row-level locking. A document database can implement transactions, but capacity and uniqueness rules would then live in application code rather than being enforced by the data layer itself.
When we'd switch	Heavy geospatial search at scale would motivate the PostGIS extension (still PostgreSQL); unstructured data such as chat history might justify a document store alongside it.

> **Note**
>
> The seat-capacity rule illustrates the distinction concretely: in PostgreSQL it is a single constraint the database enforces unconditionally -- `CHECK (seats_taken <= capacity)` -- regardless of which code path performed the write. In a document database, every code path that touches seat counts would need to re-implement that check; a single omission would allow the vehicle to be overbooked.

Decision: Database access	
We picked	**Drizzle ORM** with drizzle-kit for migrations.
Other options	Prisma, Knex, or raw pg with hand-written SQL.
Why this fits	Drizzle's query syntax closely mirrors SQL, keeping the generated statements legible. CHECK constraints, enums and partial indexes are expressed directly in the schema file, and drizzle-kit generates plain, inspectable `.sql` migration files. Row locking (`SELECT ... FOR UPDATE`) and transactions are supported directly. Prisma is comparable in ergonomics but requires hand-edited SQL for CHECK constraints and partial indexes, obscuring more of the generated query.
When we'd switch	Knex is a lower-level fallback offering the same schema design with less abstraction.

Decision: Authentication	
We picked	Custom implementation: **bcrypt** password hashing and a **JWT in an httpOnly cookie**, with role (PASSENGER / DRIVER) embedded in the token.
Other options	Auth.js / NextAuth, Clerk, Supabase Auth, or database-backed sessions.
Why this fits	Framework auth libraries for Next.js run inside the Next.js server, while business rules run in Express; splitting authentication across both would blur that boundary. A custom implementation is small (roughly 80 lines) and fully auditable. The httpOnly cookie prevents the token from being read by client-side scripts.
When we'd switch	Social login, password-reset email, two-factor authentication, or 'log out everywhere' would each justify a database-backed session table or a managed provider.

Decision: Phone verification	
We picked	**Simulated OTP**: a real `otp_codes` table (hashed code, 5-minute expiry, a 5-attempt limit, a resend limit). With `DEMO_MODE=true`, the code is returned in the response and logged instead of being sent by SMS. All delivery is routed through a single `sendOtp()` function.

Decision: Phone verification	
Other options	A paid SMS gateway, Firebase phone auth (free tier, but introduces a Google dependency and additional client complexity), or omitting verification entirely.
Why this fits	The assignment brief prohibits paid services. Omitting verification would remove a genuine safety signal, since the 'phone verified' badge shown to other passengers depends on it. Implementing the real verification flow and simulating only the delivery step preserves that signal without cost.
When we'd switch	Production deployment would substitute a regional SMS provider inside `sendOtp()` and move sending onto a background queue.

Decision: Validation	
We picked	**Zod**, defined once in a shared module used by both frontend and backend.
Other options	Joi, express-validator, or hand-written checks.
Why this fits	A single schema serves both the client (instant form feedback) and the server (a clear 400 response on invalid input), eliminating duplicate validation logic.
When we'd switch	No condition identified for this project's scale.

Decision: Real-time updates	
We picked	**Server-Sent Events** with automatic client re-fetch on reconnect, and polling as a fallback.
Other options	WebSockets (Socket.io), plain polling, or a hosted service such as Pusher.
Why this fits	Updates flow server to client only; SSE is a native browser capability (`EventSource`), reconnects automatically, and requires no additional library or infrastructure.
When we'd switch	Two-way communication (e.g. in-app chat) would justify WebSockets. Multiple API servers would require a shared message channel (Section 16).

Decision: Maps and place search	
We picked	**Leaflet** with OpenStreetMap tiles; search runs against a seeded table of roughly 50 Dhaka places, with pin drops snapped to the nearest zone.
Other options	Google Maps (requires billing), Mapbox (API token, usage limits), or a free geocoding service such as Nominatim.
Why this fits	No API key, no rate limits on search, and full functionality inside Docker with no external dependency. The assignment brief explicitly discourages spending implementation effort on mapping APIs.
When we'd switch	A production deployment would use a budgeted geocoding API and real road-network distances.

Decision: Frontend data fetching	
We picked	**TanStack Query**.
Other options	Plain fetch with useEffect, or SWR.

Decision: Frontend data fetching	
Why this fits	Provides loading, error and empty states without boilerplate, caches responses, and exposes a single call (`invalidateQueries`) to mark data stale -- exactly the primitive an SSE notification needs to trigger a re-fetch.
When we'd switch	Not warranted at a smaller scale than this project's.

Decision: Testing	
We picked	**Vitest** and **Supertest**, run against a real PostgreSQL instance in Docker.
Other options	Jest, or mocking the database layer.
Why this fits	The highest-risk code paths are database-level behaviours -- row locks and constraints -- which a mocked database cannot validate. Vitest exposes the same API as Jest (describe / it / expect) but supports ES modules without additional configuration. Supertest issues real HTTP requests against the Express app without binding a port, exercising endpoints the same way the frontend would.
When we'd switch	A frontend-heavy build would add Playwright for end-to-end coverage.

Decision: Hosting (free tier)	
We picked	**Vercel** (Next.js), **Render** free web service (Express, built from its Dockerfile), and **Neon** free PostgreSQL.
Other options	A single Render deployment, Railway (trial credits), Fly.io, or a self-hosted VPS.
Why this fits	Each option is free at this scale with no payment method required. Render's free PostgreSQL tier is deleted after 30 days, so Neon is used for the database instead. The API is built from the same Dockerfile locally and in deployment, keeping local and hosted behaviour identical.
When we'd switch	A paid tier would be adopted once real user traffic arrives, primarily to remove Render's idle sleep behaviour.

Free-Tier Constraint

Render's free web service **sleeps after 15 minutes** of inactivity; the first request after that takes roughly a minute to wake it. Documented as a known limitation and warmed prior to any live demonstration or review.

Free-tier terms change frequently and should be re-verified shortly before deployment.

Decision: Logging	
We picked	**winston** (structured JSON logs) with a request ID attached to every line via middleware.
Other options	console.log, or pino (faster, with automatic request logging via pino-http).
Why this fits	At this traffic level, pino's throughput advantage is immaterial; winston is the better-understood tool and is fully configurable and debuggable. A request ID on every line makes any given event ('what happened to ride 42?') searchable across the full request lifecycle.
When we'd switch	Multiple concurrent servers would justify routing logs to a centralised aggregation tool (Section 16).

5. Locations: Tesla Stands, Zones and the Pooling Rule

How Teslas really work in Dhaka (and why the app copies it)

Shared Teslas do not come to the passenger's door. Passengers walk to a **stand** (Arabagh mor, the Road 11 police box, the Mohakhali bus stand), the driver calls out destinations, and waits until the seats fill. The app preserves that model while removing the uncertainty: **before walking to the stand**, the passenger knows whether a Tesla is there, whether a seat is available (*seat ase?*), and what the fare will be.

> **Grounding in Existing Behaviour**
>
> This framing — walking to a known stand and waiting for seats to fill — is drawn directly from how shared Teslas already operate in Dhaka, and grounds the product in an observed behaviour rather than an assumed one.

Three layers: the pin, the stand, the zone

Layer	What it is	Used for
Exact point	The passenger's pin or searched place (lat/long + label).	Finding the nearest stand. Private rides are picked up here, at the door.
Tesla stand	A real pickup spot, 1–3 per zone (Uttara is large, Banani is small), seeded with real, named Dhaka locations.	Where **shared** rides are picked up. Pooling rule R1.
Zone	Banani, Gulshan 1, Gulshan 2, Mohakhali, Tejgaon, Farmgate, Dhanmondi, Mirpur, Uttara, Bashundhara, Badda, Motijheel.	Fare distance, drop-off rule R2, driver's area.

Picking the pickup

Nusrat searches or drops a pin. The server snaps it to the nearest stand and zone and returns something like **'Nearest stand: Banani Road 11 police box · 350 m · ~5 min walk'**. Walk time is straight-line distance × 1.3 ÷ 5 km/h, nothing fancy. If the nearest stand is more than 1 km away, the app shows a gentle warning and suggests a private ride.

Ride type	Picked up at	Why
Shared	The nearest **Tesla stand**	Pooling needs one meeting point, and it's how shared Teslas already work. Stands are public, busy places, which is also safer.
Private	The passenger's **exact pin** (door pickup)	Nobody else to meet. A real reason to choose private, beyond having the Tesla to yourself.

How search works

We seed a `places` table with ~50 Dhaka spots, each marked as a **STAND** or a **LANDMARK**, with coordinates and its zone. Search (`GET /api/places?q=banani`) is answered by our own database. Pin-drop snapping is a straight-line distance (haversine) check over a few dozen stands. No outside service, no key, no limits, no geo database.

Where is Jashim? (no GPS)

We know passengers' points because they give them to us. We **don't** track the driver. Jashim **declares his area** when he goes online ('I'm in Banani'), and after his last drop-off his area automatically becomes the zone where he ended up. Passengers see **'1 Tesla online near Banani'**, which is just a count of online drivers whose declared area is Banani.

> **Honest wording**
>
> README wording: 'Driver area is self-declared, not GPS-tracked. In production the driver app would send its location every few seconds (see the location service in the scaling section).' Optional free extra: a 'use my location' button using the browser's built-in `navigator.geolocation`, snapped to the nearest zone. Keep the manual picker for the demo, since both 'phones' are one laptop.

Distance between zones

We keep a hand-written **zone distance table** (approximate road distance in km, rounded to 0.5, looked up once on a map). 12 zones means 66 pairs. The seed script loads it into the `zone_distances` table, and a test checks every pair exists. The distances our demo depends on:

From	To	Distance
Banani	Mohakhali	3.0 km
Banani	Gulshan 1	2.0 km
Mohakhali	Gulshan 1	2.5 km
Banani	Tejgaon	4.0 km (Scenario E)
Mohakhali	Tejgaon	2.0 km (Scenario E)
Gulshan 1	Tejgaon	3.0 km (Scenario E)

> **Note**
>
> Why a table instead of a formula? Because the reviewer must check fares **by hand**. 'Look up Banani→Mohakhali in the README: 3.0 km' is checkable in 5 seconds. A formula with square roots and cosines is not.

The pooling rule (invented by us, applied the same way every time)

Jashim chooses who to take, but the app labels every waiting request as **fits** or **doesn't fit** his current ride. A request fits if all six are true (if not, every failing reason is shown):

#	Rule	Why
R1	Same **pickup stand** as the ride.	One meeting point, like a real stand. Jashim doesn't drive around collecting people.
R2	Its **drop-off zone is within 3 km** of every drop-off already in the ride (same zone counts as 0 km).	Nobody's trip gets a big detour. About a 10–15 minute detour in Dhaka traffic. 3 km lives in a config setting.
R3	Seats asked for ≤ free seats in Bullet.	Capacity.

#	Rule	Why
R4	The ride hasn't reached 'arrived' yet (status OPEN).	Once Jashim is at the pickup, the group is locked. Simple and predictable.
R5	**Women-only:** a women-only request can join a ride only if **every** passenger already in it is female. A ride that contains any women-only booking accepts only female passengers after that.	Safety, controlled by passengers. The system never decides on its own; it only enforces what a passenger asked for.
R6	**Private:** a private request can only be accepted into an empty Tesla, and a private ride accepts nobody else.	Hiring the whole Tesla means the whole Tesla.

Applying it to the story (Scenario A)

Request	Trip	Result on Jashim's screen
Nusrat, 1 seat, share	Banani → Mohakhali	First request. Accepting it creates the ride at the Banani Road 11 stand. 2 seats left.
Rafiq, 1 seat, share	Banani → Gulshan 1	**Fits.** Same pickup (R1). Gulshan 1 is 2.5 km from Mohakhali (R2 ✓). 1 seat left.
Shirin, 1 seat, women-only	Banani → Mohakhali	**Doesn't fit:** 'women-only request, this ride has a male passenger' (R5). Grey card. Mokbul can take her in Toofan (Scenario C).
Someone to Dhanmondi	Banani → Dhanmondi	**Doesn't fit:** 'Dhanmondi is 8 km from Gulshan 1' (R2).

Scenario E: three destinations from one stand

Passenger	Trip	Why it fits
Nusrat, share	Banani Road 11 stand → Mohakhali	First accept.
Rafiq, share	Banani Road 11 stand → Gulshan 1	Gulshan 1 is 2.5 km from Mohakhali.
Shirin, share (not women-only this trip)	Banani Road 11 stand → Tejgaon	Tejgaon is 2.0 km from Mohakhali and 3.0 km from Gulshan 1. Bullet is now full.

How Jashim sees requests

Jashim sees waiting requests for the stands in his declared area, grouped by stand, oldest first, with how long each person has waited. After his first accept, his list is filtered to the ride's stand.

Suggested drop-off order

Once the trip starts, Jashim's drop list is sorted **nearest-first from the pickup zone**, using the distance table we already have. In Scenario E: Rafiq (Gulshan 1, 2.0 km), Nusrat (Mohakhali, 3.0 km), Shirin (Tejgaon, 4.0 km). It's labelled 'suggested' because it isn't a real route. Jashim knows Dhaka's roads better than our table, so he can drop people in any order.

What we deliberately don't do

Idea	Why it's out	What it would need
Picking up passengers at different spots	We'd have to know where Jashim is between pickups.	Driver GPS, routing, per-passenger pickup states.
Filling a freed seat mid-trip	After Rafiq gets off, we don't know where Bullet is, so we can't tell if a new pickup is on the way. Fares would also need per-segment discounts, which the reviewer can't check by hand.	Driver GPS, route-aware matching, per-segment fares.

Both go under 'Next improvements' in the README. Our data model is already partly ready: fares, statuses and drop-offs are all per booking.

What if 8 people request and Bullet has 3 seats?

Nobody gets auto-booked into Bullet. All 8 requests go into the **Banani waiting list**. Jashim and Mokbul both see the same list and accept from it. Each accept claims seats. When Bullet is full, the remaining requests simply keep waiting for the next Tesla (or expire after 15 minutes).

Moment	What Jashim can do	Why
Before accepting	Accept or skip anyone. Skipping is just not tapping.	Nobody is in his car yet.
Accepted, not arrived yet	Cancel the **whole** ride. Everyone goes back to the waiting list.	No picking people off one by one.
Arrived	Tick who has **boarded**. Mark a passenger **No-show** only after waiting 5 minutes (their seats are freed, but the group is locked so nobody new joins). If he starts before someone boards, that person goes back to the waiting list, not to the trip.	The only legit reason to remove someone.
Started	Drop off each passenger at their stop.	Nobody can be removed mid-ride.

> **Note**
> Why no kick-out? If drivers could remove accepted passengers, they could remove someone for their looks or gender, after that person has stopped looking for other rides. So the rule is: choose freely **before**, commit **after**.

All of this updates live without refreshing: a new request nudges every online driver in Banani; when Mokbul accepts Nusrat, her card vanishes from Jashim's list within milliseconds; if both tap her at the same instant, the database picks one (Section 9).

Women-only: who turns it on?

Only passengers. The driver has no women-only switch, and the system never marks a ride women-only just because two women happen to be in it. A ride counts as women-only while it contains at least one women-only booking. That's worked out on the spot from the bookings, inside the same locked transaction as the accept, so it's always correct.

Ride currently has	New request	Result
Nobody (empty)	Shirin, women-only	Fits. From now on only women can join.

Ride currently has	New request	Result
Nusrat + another woman, neither asked for women-only	Shirin, women-only	Fits (everyone on board is female). Now no men can join.
Nusrat + Rafiq	Shirin, women-only	Doesn't fit: 'this ride has a male passenger'.
Shirin (women-only)	Rafiq	Doesn't fit: 'women-only ride'.
Shirin (women-only)	Nusrat, didn't ask for women-only	Fits. She's female.
Anyone	Private request	R6 decides (private needs an empty Tesla).

Why it works this way

Why no driver switch? Jashim already chooses who he accepts, so a switch adds nothing for him. And a driver-controlled gender filter protects no passenger. The person who needs the protection is the one who should control it. If the women-only passenger cancels or is a no-show, the restriction disappears automatically because it's worked out from whoever is still in the ride.

Only users with gender FEMALE see the women-only toggle, and the server rejects the flag from anyone else (UNDISCLOSED counts as not female for this rule). Female drivers and a 'female driver only' preference are listed as next improvements.

Who can see whom (safety + privacy)

Viewer	Sees	Never sees
Passenger	Co-riders' first name, 'phone verified ✓' badge, number of co-riders. Driver's first name, vehicle name, registration number.	Co-riders' phone, fare, exact pickup/drop point.
Driver (before accepting)	Name, stand, drop zone, seats, share/private, women-only. For private requests only 'near Banani Road 11', **not** the exact door pin.	Exact home locations of people he hasn't accepted.
Driver (after accepting)	Each of his passengers' first name, pickup (stand or exact door pin), drop zone, seats, share/private, women-only, and **fare to collect in cash**.	Passengers' fares of other rides, passwords, anything outside his rides.

Known Limitation

Gender is self-declared and not independently verified beyond the NID format check, so it can be misrepresented. Documented as a known limitation; the production fix is full NID verification against an authoritative source. Female-only driver matching and an in-app SOS control are listed under future improvements rather than addressed here.

6. Fare Model

The formula

```
distanceCharge = distance_km x 25 Tk
poolDiscount   = 25% of distanceCharge   (only if the ride starts with 2+ separate bookings)
farePerSeat    = 40 Tk (base) + distanceCharge - poolDiscount
passengerFare  = farePerSeat x seats booked
```

Nusrat and Rafiq, by hand

	Nusrat (Banani → Mohakhali, 3.0 km)	**Rafiq (Banani → Gulshan 1, 2.0 km)**
Base	40.00	40.00
Distance charge	3.0 × 25 = 75.00	2.0 × 25 = 50.00
Pool discount (25%)	− 18.75	− 12.50
Pooled fare	**Tk 96.25**	**Tk 77.50**
Fare if riding alone	Tk 115.00	Tk 90.00

These two numbers become a test: '*Nusrat's pooled fare is 9625 poysha and Rafiq's is 7750 poysha*'.

Scenario E by hand (three passengers)

Passenger	**Distance**	**Pooled fare**
Rafiq → Gulshan 1	2.0 km	40 + 50.00 − 12.50 = **Tk 77.50**
Nusrat → Mohakhali	3.0 km	40 + 75.00 − 18.75 = **Tk 96.25**
Shirin → Tejgaon	4.0 km	40 + 100.00 − 25.00 = **Tk 115.00**
Jashim collects		**Tk 288.75** cash

Nusrat's and Rafiq's fares are the same as in Scenario A: the discount depends on 'shared or not', not on how many people share.

Private hire (Scenario D)

```
privateFare = (base + distanceCharge) x the Tesla's seats     (no pool discount)
Nusrat private in Bullet, Banani -> Mohakhali:  (40 + 75) x 3 = Tk 345.00   (34500 poysha)
```

> **Private-Hire Pricing Rationale**
>
> Why not a cheaper multiplier like x1.5? Because then private becomes cheaper than sharing for any group: two friends booking 2 shared seats alone pay 2 x 115 = Tk 230, but private would be Tk 172.50 **and** come to their door. Nobody would ever book shared seats. Charging for every seat blocked guarantees private hire is never cheaper than any shared combination.

Door pickup for private rides costs nothing extra: fares use zone-to-zone distance, and the short trip from the stand to the door stays inside the zone.

> **Note**
>
> The implementation trick: when a private request is accepted, we set the ride's `seats_taken = capacity`. Our existing seat check then blocks everyone else automatically. No new locking code.

When is the fare decided?

Moment	What the passenger sees
While choosing a trip	Solo estimate (Tk 115.00) plus 'as low as Tk 96.25 if shared'.
Matched, others in the car	Live estimate updates to the pooled price.
Someone cancels or no-shows before start	**Her price does not go up.** Once she has been shown a pooled price after matching, that's her ceiling.
Trip **starts**	Fare is **locked**: the calculated fare, or her ceiling if that's lower. Saved with its full breakdown. It never changes during the ride.
Dropped off early	No change. They booked the trip; getting off earlier doesn't make it cheaper or more expensive.

> **Note**
>
> Why lock at 'start'? Before start, who's in the car can still change. After start, nobody can join or leave, so the count is final. A price that changes mid-ride would feel like a scam.

The price-never-goes-up rule

Every booking has a `fare_cap_poysha` column. It starts at the solo fare when she's matched. Whenever someone joins her ride, every booking in that ride gets `fare_cap = min(fare_cap, pooled fare)`. At start: **final fare = min(calculated fare, fare_cap)**.

Case	Calculated at start	Cap	Nusrat pays
Scenario A, both ride	Tk 96.25	Tk 96.25	Tk 96.25
Rafiq joined, then no-showed	Tk 115.00 (alone)	Tk 96.25	**Tk 96.25** (Jashim absorbs Tk 18.75)
Nobody ever joined	Tk 115.00	Tk 115.00	Tk 115.00

Without this rule, a passenger could be quoted a pooled fare, board on that basis, and then be charged the solo fare because a co-rider failed to appear — an outcome identified during risk review (Section 19, item 8) and addressed by capping the fare rather than recalculating it upward after matching.

How we store money

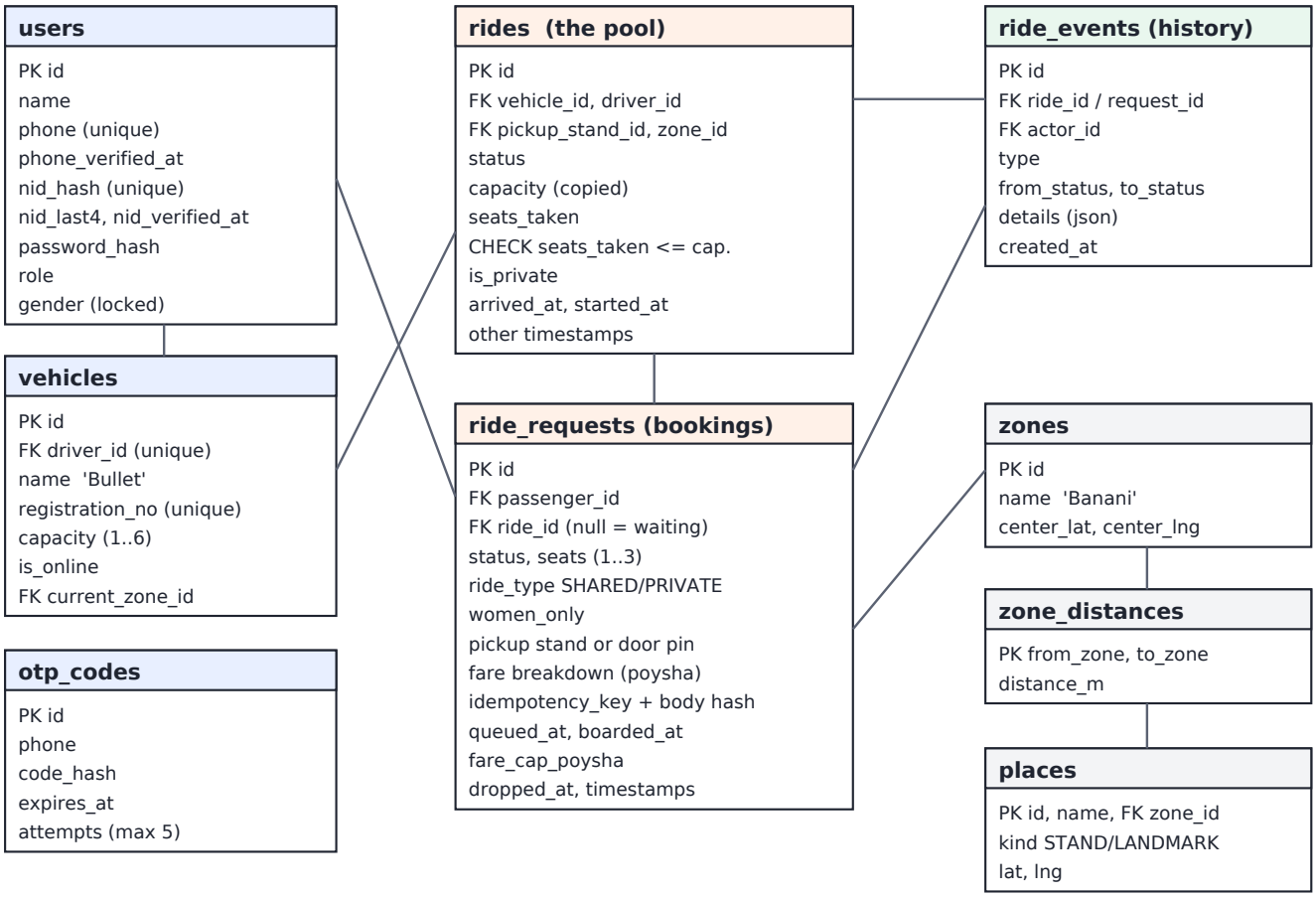
Decision: Money format	
We picked	Whole numbers of **poysha** in integer columns. Tk 96.25 is stored as 9625. Discounts are rounded **down** to a whole poysha (written in the README).
Other options	Decimal columns (NUMERIC), floating-point numbers.
Why this fits	In JavaScript `0.1 + 0.2 = 0.30000000000000004`. Money must never drift. Integers are exact in both JS and Postgres, and JS reads Postgres integers as normal numbers (NUMERIC comes back as text, which is annoying). An int column holds up to about 21 million taka per value, plenty for a rickshaw fare.
When we'd switch	Multiple currencies or tax calculations with fractions of poysha → NUMERIC with a money library.

Payment

MVP: **cash**, paid to Jashim at drop-off. The request stores `payment_method = 'CASH'` so adding a wallet later doesn't change the table shape. A simulated in-app wallet (TeslaPay) was scoped out of this build in favour of the safety and identity-verification features, which were judged higher priority within the available time.

7. Data Model

Nine tables. Each one has a clear single job. Below is the diagram, then every table explained.



Lines: one driver has one car · one car makes many rides · one passenger makes many bookings · one ride holds many bookings (that's the pool) · rides and bookings each have many events · bookings point to zones for pickup and drop · otp_codes is matched by phone number, so it needs no link.

Table by table

Table	One row means…	Important rules
users	One person who can log in: Nusrat, Rafiq, Shirin, Jashim, Mokbul.	phone is unique (login is phone + password). role is PASSENGER or DRIVER. Passwords stored only as bcrypt hashes. phone_verified_at is empty until the OTP is confirmed; unverified passengers can't request rides. nid_hash is unique (one NID, one account); only the last 4 digits are kept readable. gender is set at signup and never editable (FEMALE / MALE / UNDISCLOSED).
vehicles	One Tesla: Bullet (capacity 3) owned by Jashim. Toofan owned by Mokbul.	driver_id unique = one Tesla per driver. registration_no unique. capacity must be 1–6 (CHECK).
otp_codes	One verification code sent to one phone.	Code stored hashed, expires in 5 minutes, max 5 wrong tries. Old codes for the same phone are invalidated when a new one is sent.
zones	One area of Dhaka.	name unique.
zone_distances	Distance between two zones, stored both directions.	Primary key is the pair, so no duplicates.
places	One searchable spot: a **Tesla stand** (Banani Road 11 police box, Mohakhali bus stand…) or a landmark.	kind is STAND or LANDMARK. Belongs to a zone. Only ~50 rows, so search needs no special index.
rides	One car trip, which is the **pool**. Everyone booked into it shares Bullet.	`CHECK (seats_taken BETWEEN 0 AND capacity)`. Only one active ride per vehicle (partial unique index). pickup_stand_id is the stand for shared rides and empty for private door pickups. A ride whose last booking leaves while OPEN is cancelled automatically. is_private is set by the first accepted booking; accepting a private booking sets seats_taken = capacity, so nobody else fits. Women-only is **not** stored here: it's worked out from the ride's bookings, so there's one source of truth and nothing to keep in sync on cancel.
ride_requests	One passenger's booking: from, to, seats, status, their own fare.	seats 1–3 (a private booking's seats become the Tesla's capacity at accept). Pickup zone ≠ drop zone. queued_at resets every time the booking goes back to waiting (expiry counts from it). boarded_at is set by the driver's tick. fare_cap_poysha holds the price ceiling. Only one active booking per passenger. ride_type SHARED or PRIVATE. 'Women-only needs a female passenger' can't be a CHECK (it lives in another table), so the service enforces it and a test proves it. If status says matched/started/completed, ride_id must be set (CHECK). (passenger_id, idempotency_key) unique.
ride_events	One thing that happened: 'Rafiq joined ride 42', 'ride 42 started', with who did it and when.	Append-only: our code only inserts, never edits or deletes. This is the 'explain exactly what happened' history.

Four Notable Schema Decisions

a) Why is pool membership just a column, not its own table?

The assignment brief lists 'pool membership' as a thing to model. A booking can belong to **at most one** ride, so a `ride_id` column on ride_requests is enough: 'which bookings are in ride 42?' = all requests with ride_id 42. A separate join table is for many-to-many (a booking in several rides), which can't happen here. If we later let bookings hop between rides and wanted to remember each hop, the history already lives in ride_events.

b) Why copy capacity into rides?

A CHECK rule can only look at **its own row**. It can't peek into the vehicles table. By copying Bullet's capacity into the ride when it's created, the database can guard `seats_taken <= capacity` on the same row. Bonus: if Jashim's car were edited mid-trip, the ride keeps the rule it started with.

c) Why store seats_taken instead of adding up bookings every time?

Adding up is always correct but the database can't put a CHECK rule on a sum. A stored counter can be guarded by the database and locked (Section 9). The cost: we must update it in the same transaction as every accept and cancel. A test checks that `seats_taken` always equals the sum of active bookings' seats.

d) Why UUIDs for IDs?

IDs like 1, 2, 3 are easy to guess and reveal how many rides exist. Random UUIDs aren't guessable. We still check ownership on every request (Section 10), so this is an extra layer, not the main protection.

Indexes (the 'book index' for fast lookups)

Index	Which screen needs it
ride_requests (pickup_zone_id, pickup_stand_id, created_at) where status = 'REQUESTED'	Jashim's list of waiting requests. Only indexes waiting ones, so it stays small.
ride_requests (passenger_id, created_at desc)	Nusrat's ride history.
ride_requests (ride_id)	'Who's in this ride?'
rides (vehicle_id) unique where status in (OPEN, ARRIVED, STARTED)	Rule: one active ride per car. Also fast lookup of Jashim's current ride.
ride_requests (passenger_id) unique where status is active	Rule: one active booking per passenger.
rides (driver_id, created_at desc)	Jashim's ride history.
ride_events (ride_id, created_at), (request_id, created_at)	The replay timeline.
otp_codes (phone, created_at desc)	Finding the latest code for a phone, and counting resends.

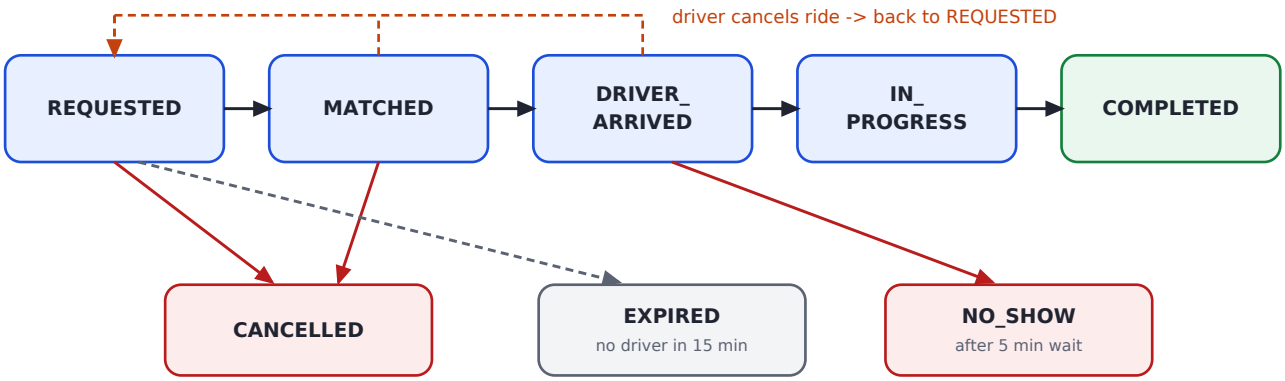
> **Note**
>
> A 'partial unique index' is a uniqueness rule that only applies to some rows. 'A driver can have many rides, but only one that isn't finished' is exactly that. The database enforces it even if two requests race.

8. Ride Lifecycle and State Machine

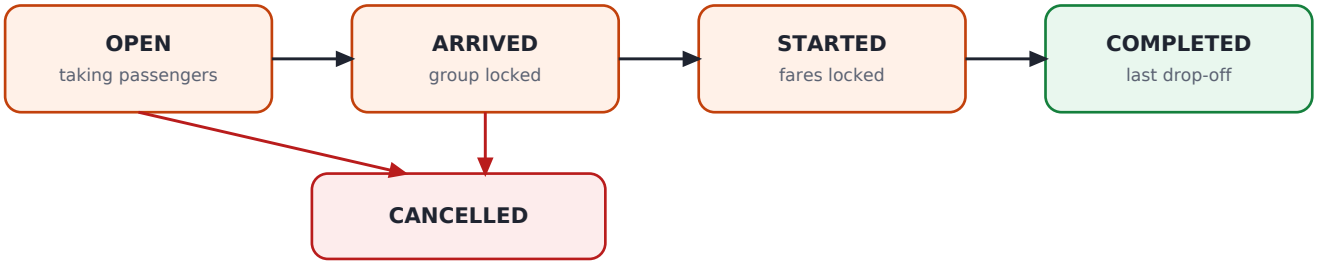
The assignment brief suggests one chain of statuses: REQUESTED → MATCHED → DRIVER_ARRIVED → STARTED → COMPLETED (+ CANCELLED). It invites us to improve it if we can explain why. Our improvement: **two linked status chains**, one for the car trip and one for each passenger's booking.

> **Note**
>
> Why two? Because in a pool, 'the ride' and 'Nusrat's booking' are different things. If Rafiq cancels, the ride is still going, but Rafiq's booking is cancelled. One status can't describe both. The assignment brief itself says each passenger must see **their own** status.

Booking status (what Nusrat sees)



Ride status (what Jashim sees)



How the two chains move together

Action (who)	Ride becomes	Bookings in it become	Allowed when
Nusrat requests	—	REQUESTED	Phone verified, no other active booking.
Jashim accepts first request	OPEN (created); is_private copied from the booking	that one: MATCHED	Jashim online with a vehicle, no active ride.
Jashim accepts another	stays OPEN	that one: MATCHED	Ride OPEN and request fits (R1–R6).
Jashim: 'arrived'	ARRIVED	all: DRIVER_ARRIVED	Ride OPEN with at least 1 booking.
Jashim ticks 'boarded'	stays ARRIVED	that one: boarded_at set	Ride ARRIVED.
Jashim: 'no-show' on one	stays ARRIVED, seats freed (auto-CANCELLED if nobody is left)	that one: NO_SHOW	Ride ARRIVED and 5 minutes since arrived_at.
Jashim: 'start'	STARTED	boarded: IN_PROGRESS, **fares locked** (price cap applied). Not boarded: back to **REQUESTED** ('driver left without you')	Ride ARRIVED with at least 1 boarded passenger.
Jashim: 'drop' on one	COMPLETED when the last one is dropped	that one: COMPLETED	Ride STARTED, booking IN_PROGRESS.
Passenger cancels	stays, seats freed; **auto-CANCELLED if it was the last booking**	theirs: CANCELLED	Their booking is REQUESTED or MATCHED, or DRIVER_ARRIVED if the ride hasn't started 15 min after arrival.
Ride STARTED for over 3 hours	COMPLETED ('auto-closed')	remaining: COMPLETED	Checked lazily on read, so a vanished driver can't trap passengers.
Jashim cancels the ride	CANCELLED	all go back to **REQUESTED** (not cancelled)	Ride OPEN or ARRIVED.
Nobody accepts in 15 min	—	EXPIRED	Checked whenever requests are listed (no background job).

Rules we decided (assumptions)

- **No kick-out.** Before accepting, Jashim can skip anyone. After accepting, he can only cancel the whole ride (before starting) or mark a no-show (after 5 minutes of waiting). Section 5 explains why.
- **Passengers can't cancel after the driver has arrived.** Jashim is standing there waiting. In production this would be a cancellation fee. Documented limitation.
- **If Jashim cancels, passengers aren't punished.** Their bookings go back to waiting so another driver can take them.
- **Per-passenger drop-off.** Everyone starts together (one pickup zone), but each passenger is dropped at their own stop. There's no separate 'complete ride' button: the ride completes

itself when the last passenger is dropped.

- **Mid-ride cancel isn't an app action.** The passenger tells the driver, who drops them early. The fare doesn't change. The driver can't cancel a started ride in the MVP (a breakdown would need partial fares; listed as a known limitation).
- **Drivers can't go offline during an active ride**, and can't go online without a vehicle.
- **Jashim can leave whenever he wants.** Like at a real stand, he doesn't have to wait until Bullet is full: 'arrived' then 'start' with 2 of 3 seats is fine.
- **After the last drop-off, Jashim's area becomes the zone where he ended up**, so his next request list is for Tejgaon, not Banani.
- **Expiry is 'lazy'.** Instead of a timer job, whenever the server lists requests it first marks any waiting request older than 15 minutes as EXPIRED. Same result, zero extra infrastructure.

Lock order (every transaction, no exceptions)

Any transaction that touches a ride locks the **ride row first** (`SELECT ... FOR UPDATE`), then its bookings. If one transaction locked a booking first and another locked the ride first, they could each wait for the other forever (a **deadlock**). Postgres would kill one and the user would see an error. With one fixed order, that can't happen. As a safety net, a deadlock error (code 40P01) is retried once.

How the code enforces it

All allowed moves live in **one small object**. Every status change goes through **one function**. No other code is allowed to set a status directly.

```
// services/rideStateMachine.js
export const RIDE_TRANSITIONS = {
  OPEN:     { ARRIVED: 'DRIVER', CANCELLED: 'DRIVER' },
  ARRIVED:  { STARTED: 'DRIVER', CANCELLED: 'DRIVER' },  // also SYSTEM if all no-show
  STARTED:  { COMPLETED: 'SYSTEM' },                     // after the last drop-off
  COMPLETED:{},  CANCELLED:{}
};
export const BOOKING_TRANSITIONS = {
  REQUESTED:      { MATCHED: 'DRIVER', CANCELLED: 'PASSENGER', EXPIRED: 'SYSTEM' },
  MATCHED:        { DRIVER_ARRIVED: 'SYSTEM', CANCELLED: 'PASSENGER', REQUESTED: 'SYSTEM' },
  DRIVER_ARRIVED: { IN_PROGRESS: 'SYSTEM', NO_SHOW: 'DRIVER', REQUESTED: 'SYSTEM',  // not boarded at start
                    CANCELLED: 'PASSENGER' },  // only if not started 15 min after arrival
  IN_PROGRESS:    { COMPLETED: 'DRIVER' },   // "drop"
  COMPLETED:{}, CANCELLED:{}, EXPIRED:{}, NO_SHOW:{}
};

// inside a transaction:
// UPDATE rides SET status = 'STARTED', started_at = now()
//  WHERE id = $rideId AND driver_id = $me AND status = 'ARRIVED'
// -> 0 rows changed?  reply 409 "Ride can't start from its current status"
```

> **Note**
>
> Notice the `AND status = 'ARRIVED'`. The database only changes the row if it's **still** in the status we expect. If Jashim double-taps 'start', the second tap finds status is already STARTED, changes 0 rows, and gets a polite error. This is called a **conditional update**, and it's the heart of how we stay safe.

Every successful change also inserts a row into `ride_events` in the **same transaction**, so the history can never disagree with the actual data.

9. Concurrency Control: The Last-Seat Problem

The problem in plain words

Scenario B. Bullet has 3 seats. Rafiq already has 2 (him and a colleague). **1 seat left.** Nusrat and Shirin are both waiting, 1 seat each. Jashim's phone lags, he taps accept on both almost together (or his phone and tablet both send it, or the network retries). Two requests arrive at the server at the same millisecond.

A naive server does this for each request:

```
1. Read ride:  seats_taken = 2, capacity = 3   ->  "1 free, OK!"
2. Write ride: seats_taken = 3
```

If both requests do step 1 before either does step 2, **both** see '1 free', both say OK, and Bullet gets 4 passengers. Worse: both write 3, so the counter even lies. This is a **race condition**.

Our fix: three layers of protection

Layer	What it does	Analogy
1. Lock the ride row	The accept runs inside a transaction that first does `SELECT ... FROM rides WHERE id = $1 FOR UPDATE`. The second request has to wait at this line until the first one finishes and commits. Then it reads the **fresh** numbers (3 of 3) and gets refused.	One bathroom key. The second person waits outside, and when they go in they see the new state.
2. Conditional updates	The booking update says `WHERE status = 'REQUESTED'` and the seat update says `WHERE seats_taken + $seats <= capacity`. If the condition isn't true at that instant, 0 rows change and we roll back.	Buying the last ticket: the counter only sells if a ticket is actually still there.
3. Database CHECK	`CHECK (seats_taken <= capacity)`. Even if our code had a bug, Postgres refuses to save 4 of 3.	A building's fire code. Doesn't matter what the manager says.

The accept, step by step

```
BEGIN;
  -- 1. take the key for this ride (others wait here)
  SELECT * FROM rides WHERE id = $rideId AND driver_id = $me FOR UPDATE;
  --    status must be OPEN, else 409

  -- 2. claim the booking only if still waiting
  UPDATE ride_requests SET status='MATCHED', ride_id=$rideId, matched_at=now()
   WHERE id = $requestId AND status = 'REQUESTED'
   RETURNING seats, drop_zone_id;
  --    0 rows -> 409 "already taken or cancelled"

  -- 3. check pooling rules R1, R2 (in JS, pure function, using fresh data)

  -- 4. add seats only if they fit
  UPDATE rides SET seats_taken = seats_taken + $seats
   WHERE id = $rideId AND seats_taken + $seats <= capacity;
  --    0 rows -> ROLLBACK, 409 "Not enough seats"

  INSERT INTO ride_events (...) VALUES (... 'REQUEST_MATCHED' ...);
COMMIT;
-- after commit: send SSE nudges to Jashim + everyone in the ride
```

What the losers see

Shirin's request stays REQUESTED (nothing changed for her, since everything rolled back). Jashim's screen shows a friendly 'That seat was just taken' message and the list refreshes. The API returns HTTP **409 Conflict** with a code like SEATS_UNAVAILABLE, so the frontend knows exactly what to show.

The second race: two drivers, one passenger

Jashim (Bullet) and Mokbul (Toofan) both tap accept on Nusrat at the same instant. Step 2 above saves us: only one `UPDATE ... WHERE status = 'REQUESTED'` can win. Postgres locks the booking row during the update, the second one re-checks, sees MATCHED, changes 0 rows, and gets 409 REQUEST_ALREADY_TAKEN.

Private rides use the same protection

Accepting Nusrat's private request runs the same transaction, with one change: the seat update becomes `SET seats_taken = capacity WHERE seats_taken = 0`. If anyone is already in the Tesla, 0 rows change and it's refused. Once it succeeds, the ride is full, so every later accept fails on the normal capacity check.

Double-tap from the passenger (idempotency)

Nusrat's internet lags, she taps 'Request ride' twice. The frontend creates one random ID (idempotency key) when the form opens and sends it with the request. The database has a unique rule on `(passenger_id, idempotency_key)`. The second copy hits that rule, and the server simply returns the booking that already exists. One tap or five, one booking. Plus the 'one active booking per passenger' rule as a second safety net.

What we'd change at bigger scale

Now (MVP)	At large scale
Row locks on one Postgres database. Fast because each ride is locked only for a few milliseconds, and different rides never block each other.	Still the right idea. Locks are per-ride, so 100k drivers don't block each other. We'd add lock timeouts, retry with backoff on 409s, and watch 'lock wait' metrics. If matching becomes automatic and very hot, move it to one matcher worker per city zone, so each zone's decisions happen one at a time in memory.

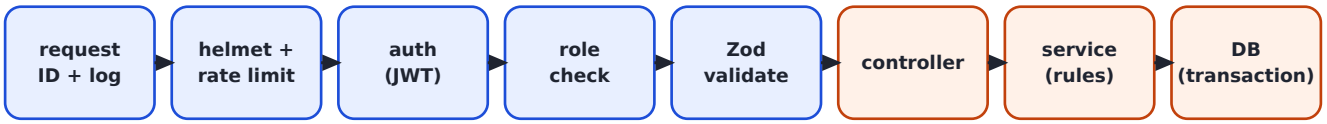
10. API Design, Error Handling and Security

Decision: API style	
We picked	**REST** with JSON. Status changes are **action endpoints** like P0ST `/driver/ride/start`.
Other options	GraphQL, one generic PATCH `/rides/:id {status}` endpoint.
Why this fits	Our data is simple and screens are fixed, so GraphQL's flexibility buys nothing. Action endpoints make every rule obvious: 'start' has its own permission check and its own allowed-from status, instead of one endpoint with a pile of if-statements.
When we'd switch	Many different clients (mobile app, partner apps) needing different shapes of data → GraphQL could help.

Endpoint list

Method + path	Who	What it does
POST /auth/signup · /auth/login · /auth/logout	anyone	Account + login cookie.
POST /auth/otp/send · /auth/otp/verify	logged in	Send (simulated) and check the phone code.
GET /auth/me	logged in	Who am I (name, role, car).
GET /places?q= · GET /zones	logged in	Search places and stands, list zones.
POST /places/nearest-stand	logged in	Pin → nearest stand, its zone, distance and walk time.
GET /zones/:id/online-count	logged in	'1 Tesla online near Banani' (drivers whose declared area is Banani).
POST /fares/estimate	passenger	Solo and pooled estimate for a trip.
POST /requests (Idempotency-Key header)	verified passenger	Request a ride: points, seats, rideType SHARED/PRIVATE, womenOnly.
GET /requests · GET /requests/:id	passenger	My history · one of my bookings (fare breakdown, timeline, co-riders' first names).
POST /requests/:id/cancel	passenger (owner)	Cancel if still allowed.
POST /driver/vehicle · GET /driver/vehicle	driver	Add / view my Tesla (name, registration, seats).
POST /driver/online · /driver/offline	driver	Toggle online, set current zone.
GET /driver/requests	driver	Waiting requests in my zone, each with fits / doesn't-fit reason.
GET /driver/ride	driver	My current ride: seats, passengers, status.
POST /driver/ride/requests/:id/accept	driver	Accept a booking (creates the ride if none).
POST /driver/ride/arrived · /start · /cancel	driver	Move the ride forward.
POST /driver/ride/requests/:id/no-show · /drop	driver	Mark one passenger no-show (after 5 min) · drop one passenger off.
GET /driver/history	driver	My past rides.
GET /events/stream	logged in	SSE connection for live nudges.
GET /health	anyone	Server + DB alive?
POST /dev/seat-race · POST /dev/scenario/:name	DEMO_MODE=true + DEMO_KEY header	Race demo (Section 12) · reset and set up Scenario A–E.

How a request travels through the backend



Folder	Job	Rule
routes/	Map URL to controller, attach middleware.	No logic.
controllers/	Read request, call a service, send response.	Thin. No SQL.
services/	All business rules: matching, fares, state machine, cancellations.	Where the thinking lives. Fare and matching are **pure functions** (no database), so tests are easy.
db/	Drizzle schema, migrations, seed.	Only place that knows table shapes.
middleware/	auth, requireRole, validate(zodSchema), errorHandler.	Reusable guards.
realtime/	SSE connections and nudge sending.	Only sends after a transaction commits.
shared/ (root)	Zod schemas + constants used by both apps.	One definition of valid data.

Errors: one shape everywhere

```
{ "error": { "code": "SEATS_UNAVAILABLE",
             "message": "Only 1 seat left in Bullet, this request needs 2." } }
```

HTTP code	When
400	Bad data (Zod failed). Includes which field.
401	Not logged in.
403	Logged in but wrong role (a passenger calling a driver endpoint), or PHONE_NOT_VERIFIED.
404	Not found **or not yours**. Rafiq asking for Nusrat's booking gets 404, not 403, so he can't even learn it exists.
409	Conflict with current state: wrong status, seat taken, request already accepted, WOMEN_ONLY_MISMATCH, NO_SHOW_TOO_EARLY.
429	Too many login attempts.
500	Our bug. Logged with request ID; user sees a generic message.

A custom `AppError(status, code, message)` class is thrown from services, and one error-handler middleware turns it into the JSON above.

Security basics (and how each is done)

Threat	Our protection
Stolen passwords	bcrypt hashes (cost 10). Plain passwords never stored or logged.
Token theft by scripts	JWT in an **httpOnly**, **Secure**, **SameSite=Lax** cookie. Expires in 7 days.
Fake or duplicate accounts	Phone numbers normalised to one format; NID stored as a unique hash; gender locked after signup.
Forged login tokens	The API refuses to start in production if JWT_SECRET is missing or under 32 characters (env checked with Zod at startup). Sessions slide: each request renews the cookie.
Secrets in logs	Request bodies are never logged; password, code and nid are redacted from anything that is.
Home addresses leaking	Private door pins are shown only to the driver who accepts; others see 'near Banani Road 11'.
Rafiq touching Nusrat's ride	Every query includes the owner: `WHERE id = $id AND passenger_id = $me`. Driver actions include `driver_id = $me`. Tested.
Passenger calling driver actions	requireRole('DRIVER') middleware.
Fake prices from the browser	The server never accepts a fare from the client. It recalculates from zones.
Bad input / SQL injection	Zod on every body/query, Drizzle uses parameterised queries.
Password guessing	Rate limit on /auth routes (express-rate-limit).
OTP guessing / SMS abuse	Codes hashed, 5-minute expiry, max 5 tries, max 3 sends per phone per 15 minutes. (With real SMS, bots spamming 'send code' costs real money.)
Unsafe pooling	Women-only rule enforced on the server inside the accept transaction, never just hidden in the UI. Co-riders see first names only.
Common header attacks	helmet middleware. CORS locked to our frontend URL.
Leaked secrets	.env in .gitignore, only .env.example committed. Render/Vercel/Neon secrets set in their dashboards.

11. Real-Time Updates and Frontend Architecture

How SSE works in our app

- When any page loads, the browser opens `new EventSource('/api/events/stream')`. The login cookie comes along automatically.
- The server keeps a list in memory: user ID → open connections.
- After a transaction **commits**, the service calls `notify([jashimId, nusratId, rafiqId], {type:'ride.updated', rideId})`.
- The browser receives the nudge and tells TanStack Query 'booking data is stale'. It re-fetches through the normal API, which applies privacy rules.
- Every 25 seconds the server sends a tiny heartbeat so proxies don't close the connection.
- If the connection drops, EventSource reconnects by itself, and we re-fetch everything once on reconnect in case we missed a nudge.
- Fallback: if no SSE message arrives for 30 seconds, queries poll every 5 seconds until it's back.

> **Honest limitation**
>
> The in-memory connection list only works with **one server**. That's fine for the MVP and a great interview point: with two servers, Nusrat might be connected to server A while the update happens on server B. Fix: Postgres LISTEN/NOTIFY (no new tools) or Redis pub/sub. Put this in Known Limitations.

Screens

Route	Who	What's on it
/login, /signup, /verify	all	Phone + password + NID + gender (locked, shown with a warning), then the OTP step (code shown on screen in demo mode). Demo login buttons ('Log in as Nusrat') for the reviewer.
/ride	passenger	Map + search boxes for pickup and drop, 'nearest stand · 350 m · ~5 min walk', Teslas online nearby, seat picker, Share / Private switch (private = door pickup), Women-only switch (female passengers), solo, pooled and private estimates, Request button.
/ride/[id]	passenger	Live status stepper, driver + Tesla, co-riders' first names with 'verified' badges, fare receipt, timeline, cancel button (only when allowed).
/history	passenger	Past bookings with final fares.
/driver/vehicle	driver	Vehicle-registration form. Presented first if no vehicle is on file.
/driver	driver	Online toggle + zone. Waiting requests with fits / doesn't-fit reasons and waiting time. Current ride: 3 seat icons, passenger first names and drop zones, next-action button, No-show buttons with a 5-minute countdown, then the drop-off list in suggested order with each passenger's cash fare and a Drop button.
/driver/history	driver	Past rides with passengers and total earned.
/dev/seat-race	demo	The Seat Race page.

Every screen handles four states

State	Example
Loading	Skeleton cards while Jashim's request list loads.
Empty	'No one is waiting in Banani right now. You'll see requests here instantly.'
Error	'Couldn't load requests. Retry' + the server message for 409s ('That seat was just taken').
Success	The actual data.

Frontend folders

```
web/
  app/                  pages (login, ride, ride/[id], history, driver, dev/seat-race)
  components/           MapPicker, PlaceSearch, StatusStepper, FareReceipt,
                        RideTimeline, SeatIcons, RequestCard, EmptyState, ErrorState
  lib/api.js            one fetch wrapper: adds JSON, throws AppError-shaped errors
  lib/queries.js        TanStack Query hooks (useMyBooking, useDriverRequests...)
  lib/useLiveUpdates.js  EventSource + invalidate on nudge
```

Leaflet + Next.js trap

Leaflet touches `window`, which doesn't exist on the server. Load the map component with `dynamic(() => import('./MapPicker'), { ssr: false })` or it will crash on build. Classic Day 3 time-waster.

12. Differentiating Features

None of these break the assignment brief. Each one takes a requirement and pushes it one step further in a way the reviewer will remember. Build 1–8. Keep 9 minimal. The TeslaPay wallet from v1 is dropped to pay for 2, 3, 7 and 8.

#	Feature	What it is	Which requirement it amplifies	Cost
1	**Seat Race**	Page with one button. Server fires two accepts for the last seat at the same moment (Scenario B), then shows: winner, loser's 409 message, final seats 3/3, and the event log.	The primary concurrency requirement.	~2 h
2	**Women-only rides and co-rider visibility**	Rule R5 on the server; co-riders shown by first name + verified badge.	Our own addition: real Dhaka safety problem.	~2.5 h
3	**Private hire**	Hire the whole Tesla. Rule R6, reuses the capacity check.	'Share when it makes sense.'	~1 h
4	**Fare receipt with the math**	Passenger sees '40.00 + 3.0 km × 25 − 25% pool = Tk 96.25'.	'Fares must be verifiable by hand.'	~1 h
5	**Fits / doesn't-fit reasons**	Jashim sees why each request can or can't join.	'Document a matching rule and apply it consistently.'	~1 h
6	**Double-tap proof booking**	Idempotency key (Section 9).	Real-world Dhaka network thinking.	~45 min
7	**Simulated OTP**	Full verification flow, SMS faked.	Sign-up done properly without paying.	~1.5 h
8	**Per-passenger drop-off + no-show**	Drop buttons; 5-minute no-show countdown.	'Jashim wants to know who's riding and when he can go.'	~2 h
9	**Ride replay timeline**	Every event from ride_events as a vertical timeline.	'Hold enough history to explain what happened.'	~1 h (keep simple)

> **Note**
>
> The Seat Race tool is the strongest artefact for demonstrating the concurrency guarantee: a brief explanation followed by one click reproduces, live, the exact failure mode this design is built to prevent.

Things we're deliberately NOT building

- Real road routing, live GPS tracking of the car, push notifications, chat, ratings, admin panel, wallet.
- Safety extras (listed as next improvements): NID verification, female drivers option, SOS button, share-trip link.
- Driver GPS tracking, pickups at several spots, and refilling a seat mid-trip (all need live location and routing; see Section 5).

- Redis, queues, microservices. Section 16 explains the conditions under which each would be introduced.

13. Testing Strategy

The assignment brief says 'meaningful, not coverage-chasing'. So we test the rules that would hurt real people if broken, against a **real Postgres** (a separate test database in Docker, wiped before each test file).

Brief's requirement	Our test (using the cast)	Type
Capacity can never be exceeded	Scenario B: Bullet has Rafiq (2) + Nusrat (1) = 3/3; Jashim accepts Shirin → 409, seats_taken stays 3.	API + DB
Capacity (database level)	Directly try to set seats_taken = 4 on Bullet's ride → Postgres CHECK error.	DB
Invalid transitions rejected	Start a ride that is OPEN (skipping arrived) → 409. Complete an ARRIVED ride → 409. Every cell of the transition table checked.	Unit + API
Pooled fares correct	Nusrat = 9625 poysha, Rafiq = 7750 poysha after start. Solo Nusrat = 11500.	Unit + API
Can't touch others' rides	Rafiq GET / cancel Nusrat's booking → 404. Mokbul tries to start Jashim's ride → 404. Nusrat calls driver endpoint → 403.	API
Cancellation rules	Nusrat cancels while MATCHED → OK and seats freed. After DRIVER_ARRIVED → 409. Jashim cancels → bookings back to REQUESTED.	API
Concurrent requests	Last seat: Promise.all of two accepts → exactly one 200, one 409, seats 3/3. Two drivers, one booking → one wins. Repeated 20×.	API + DB
Idempotency	Same Idempotency-Key twice → same booking ID, one row.	API
Women-only (ours)	Scenario A: Shirin's women-only request can't join Bullet with Rafiq (409 WOMEN_ONLY_MISMATCH). Women-only can join an all-female ride. Scenario C: Rafiq can't join Toofan's women-only ride.	API
Private (ours)	Scenario D: Nusrat private in Bullet = 34500 poysha; then accepting Rafiq into that ride → 409. Private can't join a non-empty ride.	API
No-show + drop-off (ours)	No-show before 5 minutes → 409 NO_SHOW_TOO_EARLY. Drop Nusrat → ride stays STARTED; drop Rafiq → ride COMPLETED.	API
Phone + NID (ours)	Unverified passenger can't request (403). Expired code and 6th wrong attempt rejected, including 20 parallel guesses. Same NID twice → 409. Gender change → rejected.	API
Price cap (ours)	Rafiq joins then no-shows: Nusrat pays 9625, not 11500.	API
Boarded (ours)	Start with Shirin not boarded → Shirin back to REQUESTED, not IN_PROGRESS.	API
Stuck states (ours)	Last passenger cancels → ride auto-cancelled; Jashim can go offline. Arrived 15+ min without start → passenger can cancel.	API
Seat invariant	After every action above, seats_taken = sum of seats of MATCHED / DRIVER_ARRIVED / IN_PROGRESS / COMPLETED bookings.	API + DB
Matching rule	Nusrat + Rafiq fit; Dhanmondi request doesn't, with the right reason. A request from a different stand doesn't fit (R1).	Unit

Brief's requirement	Our test (using the cast)	Type
Stands (ours)	Banani Road 11 pin snaps to the Banani Road 11 stand with the right walk distance. Private booking keeps the door pin; shared booking stores the stand.	Unit + API
Scenario E (ours)	Three drops: fares 7750 / 9625 / 11500 poysha; suggested order Rafiq → Nusrat → Shirin; ride completes after Shirin; Jashim's area becomes Tejgaon.	API
Seed data sanity	All 66 zone pairs have a distance.	Unit

Frontend: no big test suite. The risk lives in the backend. We say so in the README (a trade-off, clearly explained). If time allows, one smoke test that the pages render.

14. Docker and Deployment

Local: one command

```
docker compose up --build
  db   postgres:16      healthcheck: pg_isready
  api  ./api            waits for db healthy -> runs migrations -> seeds base world (skips what exists)
                        -> starts on :4000   healthcheck: GET /health
  web  ./web            waits for api healthy -> starts on :3000
                        /api/* rewritten to http://api:4000
open http://localhost:3000   ->  "Log in as Nusrat" / "Log in as Jashim"
```

File	Purpose
.env.example	Every variable with fake values: DATABASE_URL, JWT_SECRET, CLIENT_ORIGIN, API_URL, DEMO_MODE (shows OTP on screen, enables Seat Race), POOL_MAX_DROP_KM=3, REQUEST_EXPIRY_MIN=15, NO_SHOW_WAIT_MIN=5, OTP_EXPIRY_MIN=5.
api/Dockerfile	Small Node 20 image. Same file used by Render, so local = online.
web/Dockerfile	Next.js production build for local compose (Vercel builds it itself online).
docker-compose.yml	The three services above + a named volume so data survives restarts.
docker-compose.test.yml (or a test profile)	Separate test database, so automated tests never overwrite demo data.

Idempotent Seeding

Seed must be **safe to run twice** (it checks 'does Jashim already exist?'). Otherwise the second `docker compose up` crashes on the unique phone rule, which is a realistic occurrence since containers are commonly restarted during review.

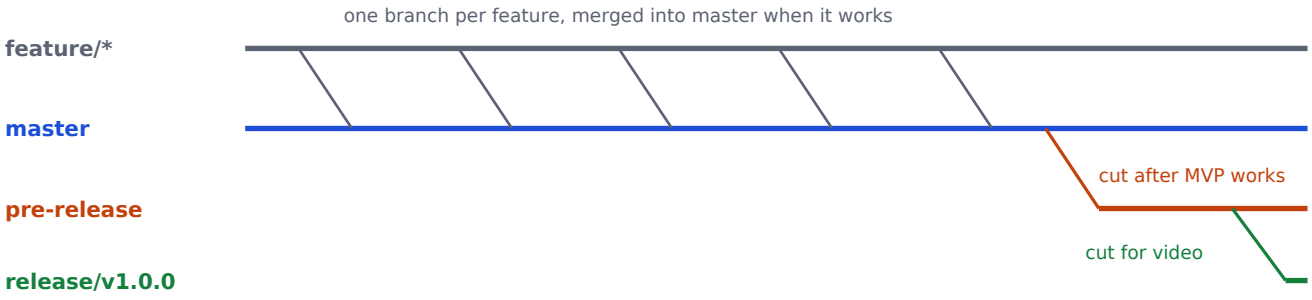
Online: free

Piece	Where	Notes
Frontend	Vercel (Hobby)	Set API_URL env to the Render URL so rewrites point there.
API	Render free web service, built from api/Dockerfile	Sleeps after 15 min idle; ~1 min to wake. Runs migrations + seed on start.
Database	Neon free Postgres	Render's free Postgres is deleted after 30 days, Neon's isn't.

Deployment verification should confirm that SSE traffic passes through the Vercel rewrite correctly. If it is buffered or terminated by the proxy, the polling fallback keeps the application functional; the observed behaviour is recorded as a known limitation.

15. Version Control Strategy

Branches



Branch	Contains
feature/project-setup	Monorepo folders, compose, Dockerfiles, lint, .env.example.
feature/db-schema	Drizzle schema, first migration, seed with the cast and Dhaka zones/places.
feature/passenger-auth	Signup/login/logout/me, JWT cookie, roles, rate limit.
feature/phone-otp	otp_codes table, send/verify, demo mode display.
feature/vehicle-onboarding	Driver adds his Tesla; online requires a vehicle.
feature/places-and-fare	Zones, places search, pin snapping, fare service + tests.
feature/ride-requests	Request, idempotency, history, cancel, expiry.
feature/driver-flow	Online/offline, request list, state machine, arrived/start, drop-off.
feature/tesla-pooling	Accept with locks, matching rules R1–R4, capacity, concurrency tests.
feature/private-hire	Rule R6, private fare.
feature/women-only-safety	Gender field, rule R5, co-rider view.
feature/dropoff-noshow	Per-passenger drop-off, no-show, auto-complete.
feature/realtime-sse	Event stream + nudges.
feature/passenger-ui, feature/driver-ui, feature/map-search	Screens.
feature/seat-race-demo	The demo page.
pre-release	Deployment fixes, README, diagrams, screenshots.
release/v1.0.0	Exactly what the video shows. Tag it v1.0.0.

Good commit messages (examples to copy the style of)

```
chore(setup): scaffold api and web workspaces
build(docker): add compose setup for api, web and postgres
feat(db): add rides and ride_requests tables with capacity check
feat(seed): seed Jashim, Bullet, Nusrat, Rafiq, Shirin and Dhaka zones
feat(auth): add passenger login endpoint with httpOnly cookie
feat(fare): calculate pooled fare in poysha
test(fare): verify Nusrat and Rafiq pooled fares
feat(pool): lock ride row when accepting a request
fix(pool): prevent overbooking when two accepts race
test(pool): prove last seat goes to exactly one request
feat(auth): add simulated phone OTP verification
feat(safety): enforce women-only pooling rule
feat(pool): support private hire of a whole Tesla
feat(ride): add per-passenger drop-off and no-show
feat(realtime): push ride update nudges over SSE
docs(readme): add ERD and concurrency explanation
```

Each feature branch is merged into master with `git merge --no-ff` once its tests pass, so the merge itself is visible in history rather than fast-forwarded away. Target roughly 3–8 substantive commits per branch.

Build Timeline

Stage	Scope	Exit criteria
1 — Foundation	Project setup, Docker, schema, migrations, seed data, authentication, OTP, vehicle onboarding. Architecture and ERD documented.	`docker compose up` starts all three services; signup, verification and login are exercisable via curl or Postman.
2 — Core logic	Places and fare service, ride requests, driver flow, state machines, locked accept transaction (R1–R6), private hire, women-only, cancellation, no-show, drop-off, full backend test suite.	Every test in Section 13 passes. This stage carries the highest correctness risk and is completed before frontend work begins.
3 — Client	SSE integration, passenger and driver screens, map and search, OTP screen, Seat Race demo tool, loading/empty/error states.	All five scenarios (A–E) are reproducible end to end across two concurrent sessions.
4 — Release	Deployment, README, diagrams, screenshots, AI-usage disclosure. release/v1.0.0 tagged and demo video recorded.	A reviewer can clone the repository, run it, open the deployed link, read the README and watch the video without further input.

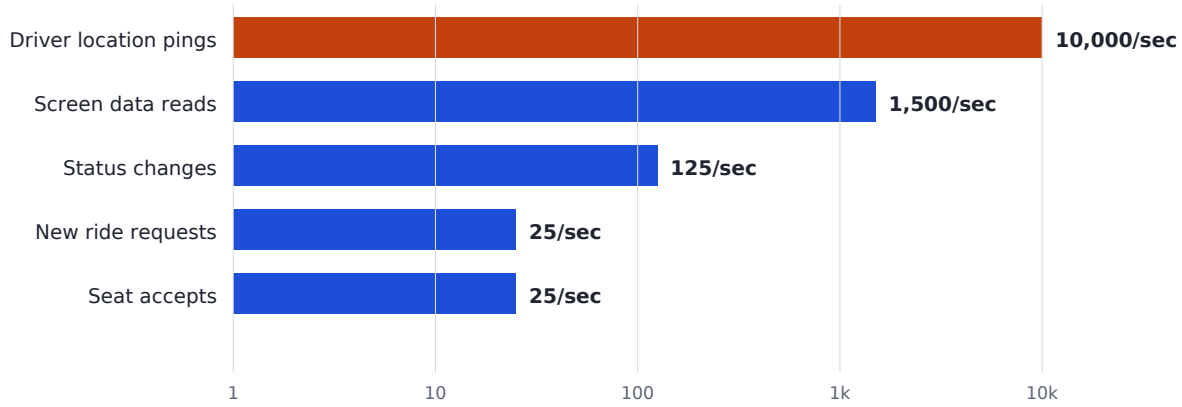
> **Scope Reduction Plan**
>
> If Stage 2 overruns, scope is reduced before correctness is: the ride-replay timeline, the OTP verification screen (the backend remains), and the no-show countdown UI are the first candidates to drop. Automated tests, documentation and the demo video are never reduced, since the assessment weights process and correctness above feature count.

16. Scalability: Growth to 1M Passengers

The assignment brief says **don't** build scaling tools into the MVP (no Redis, queues, Kubernetes). They want the **reasoning**. So our approach has two parts: build the MVP simple but with 'seams' (places where a bigger part can be swapped in later without a rewrite), and show on paper exactly how it grows, with numbers.

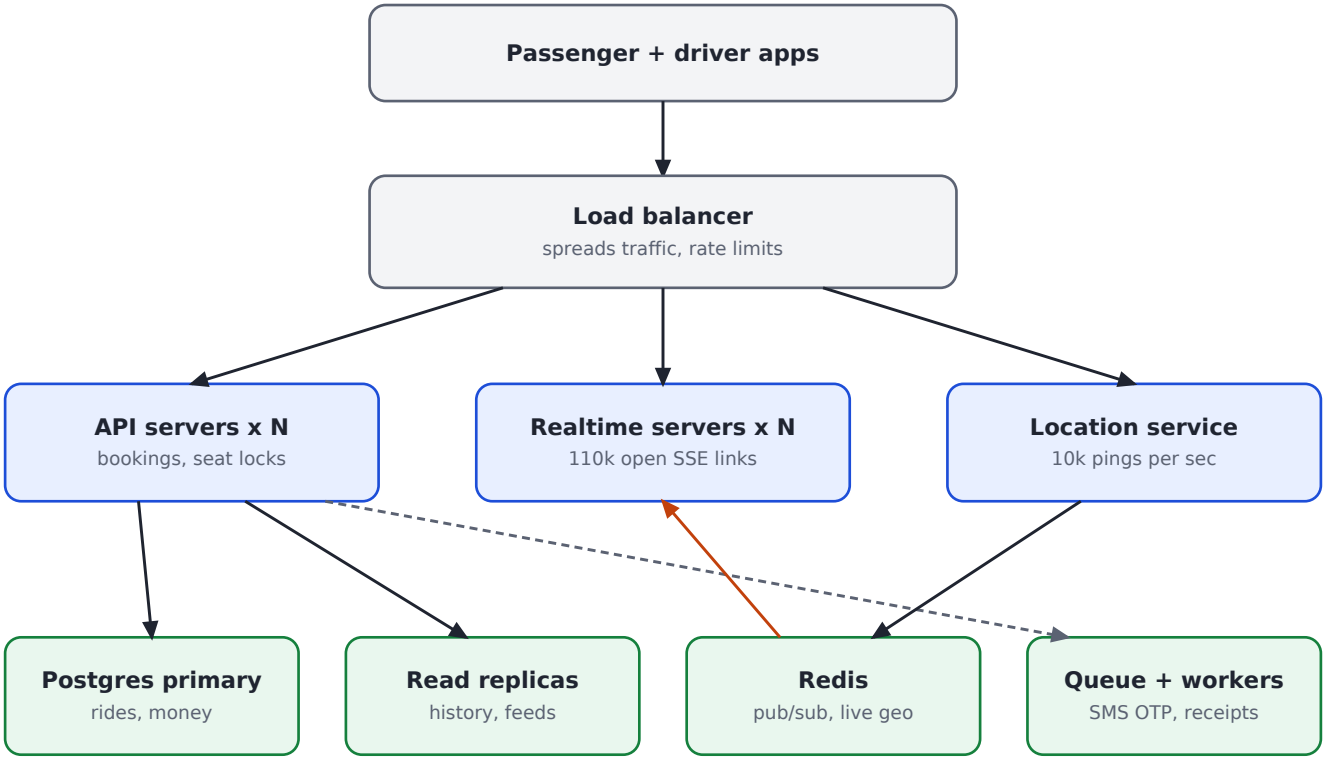
Step 1: do the math before drawing boxes

Who	Assumption	Result at the worst rush-hour minute
1M passengers	15% ride on a weekday = 150k bookings/day; half happen in 4 rush hours = ~5/sec, spiking 5×.	**~25 new requests/sec**, ~25 accepts/sec
Status changes	About 5 status changes per booking.	**~125/sec**
Screen reads	Every nudge makes a few screens re-fetch.	**~1,500/sec**
100k drivers	Half online at rush hour = 50k, each phone sends its location every 5 seconds.	**~10,000 location pings/sec**
Live connections	50k drivers + ~60k passengers waiting or riding.	**~110,000 open SSE connections**



Operations per second at peak (log scale: each grid line is 10× the previous).

Step 2: the architecture at 1M users



Blue = stateless servers (add copies freely). Green = where data lives. Orange arrow = Redis pushes 'ride changed' to realtime servers.

Piece	Plain-language job	Why it's needed
Load balancer	Traffic police: sends each request to a free server; blocks spammers.	One server can't take all traffic. Rate limiting matters extra for OTP, where bots spamming 'send code' cost real SMS money.
API servers × N	Copies of our exact Express app.	Works because our servers remember nothing between requests (login is in the JWT, data is in Postgres). Need more? Add copies.
Realtime servers × N	Hold the SSE connections, separate from the API.	110k open connections shouldn't slow down bookings.
Redis pub/sub	Group chat for servers.	Nusrat is connected to realtime server #3 but Jashim's accept ran on API server #7. #7 posts 'ride 42 changed'; every realtime server hears it and nudges its own users.
Location service + Redis geo	Stores only each driver's latest location, with built-in 'find within 2 km' search.	10k writes/sec would choke Postgres, and we don't need old locations. Our 12 zones become a grid of small hexagon cells (H3).
Postgres primary	All writes: bookings, seat locks, fares.	The **same locking code as the MVP**. Locks are per ride, so drivers never block each other.
Read replicas	Read-only copies.	History pages and driver feeds read from here, keeping the primary free.
Queue + workers	To-do list for slow jobs.	SMS, receipts, analytics happen in the background so the user never waits.

Step 3: grow in stages, only when a number says so

Stage	Users	What we add	The trigger
0 (MVP)	Demo	1 API, 1 Postgres	—
1	~10k	2–3 API copies behind a load balancer, bigger DB, Postgres LISTEN/NOTIFY to share nudges between servers (no new tools)	API CPU stays above 70%
2	~100k	Redis (pub/sub, rate limits), read replica, queue for SMS	DB read load high; SSE servers full
3	~1M	Location service with Redis geo + hexagon cells, separate realtime tier, matching split by zone	Location writes choke the DB
4	Multi-city	Separate database per city (sharding)	One primary can't take the writes

> **Note**
>
> Stage 1 needs no new tools: Postgres can pass messages between servers itself (LISTEN/NOTIFY). Knowing the cheapest next step instead of jumping straight to Redis is exactly the judgement the assignment brief is testing.

Step 4: the seams already in our MVP

MVP choice	Why it scales later
JWT in a cookie, no server memory	Any number of API copies works on day one.
Seat lock is **per ride**, not global	100k drivers never wait on each other; only people fighting over the same Tesla do.
All nudges go through one `notify()` function	Swap in-memory → LISTEN/NOTIFY → Redis by changing one file.
All SMS goes through one `sendOtp()` function	Plug in a real provider and move it onto a queue without touching auth.
Zones as the matching unit	Zones become the way work is split: one matcher per zone, one shard per city.
Idempotency keys	Retries are safe; at scale, networks fail constantly.
`ride_events` history table	Can feed analytics or event streams later without touching booking logic.
Indexes on hot queries	Driver feed and history stay fast as tables grow.
Config values (3 km, 15 min, 5 min)	Tune per city without redeploying code.

Step 5: when things fail (they will)

Failure	What happens
A server dies mid-accept	The transaction never committed, so Postgres rolls it back. No half-booked seat.
Redis dies	Realtime falls back to polling from Postgres. Slower, still works (our MVP fallback, reused).
DB primary dies	A replica is promoted automatically (managed databases do this).
SMS provider down	OTP jobs wait in the queue and retry with growing delays; a circuit breaker stops hammering a dead provider.
Bad deploy	Rolling deploys (a few servers at a time) and migrations that work with both old and new code.
Nobody notices a problem	Central logs with request IDs, metrics (p95 latency, 409 rate = seat fights, lock waits), traces, alerts.

The throughput table, chart, architecture diagram and staged-growth table above are the core artefacts for a scalability discussion: booking volume is small at any realistic scale, while location pings and open connections are what would actually require new infrastructure, introduced in the order the stages table describes.

17. Design Rationale: Common Questions

The following captures the reasoning behind decisions that are most likely to be probed in review, phrased as question and answer for quick reference.

Question	Answer
Why Postgres and not Mongo?	Relationships and seat safety. The database itself enforces capacity, uniqueness and valid links, so bugs and races can't break them.
Walk me through the last-seat problem.	Lock the ride row, re-check fresh numbers, conditional update, CHECK constraint as a final guard. Loser gets a 409 and nothing changes for them.
Why not just check seats in JavaScript before saving?	Because two requests can both check before either saves. The check and the save must be one atomic step inside the database.
Why two status chains?	A pool has one car trip but several bookings; one passenger cancelling shouldn't change the ride's status.
Why lock the fare at start?	Before start, the group can change; after start, it can't. Prices shouldn't change mid-ride.
How does Rafiq not see Nusrat's fare?	Every query is scoped to the logged-in user, and the SSE message contains no data, just a nudge to re-fetch through the API.
Why integer poysha?	Floats drift (0.1 + 0.2). Integers are exact in JS and Postgres.
Why JavaScript and not TypeScript?	See Section 4: runtime validation via Zod plus database constraints covers the risk that matters here; JSDoc restores editor tooling without TypeScript syntax.
Why REST and not GraphQL?	Fixed screens, simple data, and action endpoints make rules explicit.
What happens if the API crashes mid-accept?	The transaction never committed, so Postgres rolls it all back. No half-booked seat.
What would you change first for production?	Per-passenger drop-offs, driver location tracking, multi-server SSE, payments, no-show handling.
Show me where X is enforced.	Know the file: stateMachine.js, poolService.accept, fareService, the schema CHECKs, the auth middleware.
Why can't the driver kick someone out?	Choose freely before accepting, commit after. Otherwise drivers could remove people for their looks or gender after they stopped looking for rides. No-show after 5 minutes is the one exception.
How is women-only enforced, and what is its weakness?	Only passengers turn it on; there is no driver switch. Rule R5 is checked inside the accept transaction after the ride row is locked, using the bookings already in the ride. The weakness is that gender is self-declared; the production fix is full NID verification.
Why simulate OTP?	Real SMS costs money and the assignment brief forbids paying. The table, expiry, attempts and rate limits are real; only delivery is faked, behind one function.
Why require a walk to a stand for shared rides?	This mirrors how shared Teslas already operate in Dhaka. Pooling requires one meeting point; stands are public and busy, which is also safer. Private rides receive door pickup.
How do you know where drivers are?	We don't track GPS. Drivers declare their area, and it updates to their last drop-off zone. In production, phones would send location to a location service (Section 16).

Question	Answer
Why can't a freed seat be filled mid-trip?	We don't know where the Tesla is mid-trip, so we can't tell if a new pickup is on the way, and per-segment fares couldn't be checked by hand.
How does this scale to 1M users?	The booking data is tiny (~25/sec); location pings (~10k/sec) and open connections (~110k) are what break. Scale those, in stages, when metrics say so.
Change the pool radius from 3 km to 2 km, live.	It's one config value (POOL_MAX_DROP_KM). Show the test that now fails for Mohakhali↔Gulshan 1 (2.5 km) and explain why.

18. Assumptions

#	Assumption	Why
A1	'Tesla' = a three-wheeled battery rickshaw; drivers register their own vehicle without document checks.	Matches the story; verification is out of MVP scope.
A2	Drivers choose which requests to accept; the app shows whether each fits and why.	Keeps the driver in control; rules still enforced by the server.
A3	No kick-out: skip before accepting, whole-ride cancel before starting, no-show only after 5 minutes waiting.	Protects passengers from unfair removal.
A4	Dhaka is modelled as 12 zones with a hand-made distance table.	Brief says no real routing; makes fares hand-checkable.
A5	Pool rule: same pickup zone, drop-offs within 3 km of each other, enough seats, ride not yet arrived.	Small detours for everyone.
A6	Women-only is a passenger choice. A women-only request joins only all-female rides, and blocks men from joining after it. Drivers have no women-only switch. Gender is self-declared.	Safety stays in the hands of the person it protects; limitation documented.
A6b	Identity is backed by simulated NID verification; gender is set at signup and never changes. Companions on multi-seat bookings are taken on trust.	Benefit of the doubt, documented.
A7	Private hire takes the whole Tesla; fare = seat fare × the Tesla's seats.	Sharing is optional; private must never be cheaper than sharing.
A8	Phone verification is simulated; codes are shown on screen in demo mode.	Brief forbids paid services.
A9	The group locks when the driver arrives; fares lock at start; early drop-off doesn't change the fare.	Predictable and fair.
A10	Passengers can cancel only before the driver arrives; the driver can't cancel a started ride.	Protects the driver's time; breakdowns listed as a limitation.
A11	Each passenger is dropped at their own stop; the ride completes after the last drop.	How pooling really works.
A12	Max 3 seats per booking; one active booking per passenger; one Tesla per driver.	Matches Bullet; prevents spam and confusion.
A13	Unanswered requests expire after 15 minutes.	No one waits forever.
A14	Cash payment; money stored in integer poysha.	Brief allows it; exact maths.
A15	The assignment's last-seat race happens through the driver's accept (Scenario B), since drivers choose riders.	Consistent with A2; same database protection either way.
A16	Shared rides pick up at the nearest Tesla stand (1–3 per zone); private rides pick up at the door.	Mirrors how shared Teslas operate in Dhaka today.
A17	Driver area is self-declared, not GPS-tracked; it updates to the last drop-off zone.	No GPS or routing in scope.
A18	No pickups at several spots and no mid-trip seat refills.	Both need live driver location and routing.

19. Risk Analysis and Mitigations

Prior to implementation, the design was reviewed adversarially across two passes to surface edge cases, internal contradictions and states the system as specified could not recover from. Known, accepted limitations (no GPS, no real SMS delivery, no live routing) are out of scope for this exercise and are not repeated here; each item below is a defect in the design as written, not a limitation accepted in advance.

Each row says what breaks, why it matters, how we fix it, and the phase of the implementation roadmap (Section 20) where the fix is addressed. P0 = ground, P1 = Dhaka data, P2 = Jashim, P3 = Nusrat, P4 = where-to and fares, P5 = requests, P6 = the pool, P7 = the trip, P8 = live updates, P9–P13 = screens, P14 = ship.

A. Places where our own rules contradicted each other

#	What breaks	Why it matters	How we fix it	Phase
1	The seat check 'seats_taken = sum of active seats' fails for private rides (1 seat booked, 3 taken) and after no-shows.	Our own promised test would fail, and we couldn't trust seat counts.	At accept, a private booking's seats become the Tesla's capacity. A no-show frees its seats. The invariant counts MATCHED / DRIVER_ARRIVED / IN_PROGRESS / COMPLETED.	P6, P7
2	If everyone cancels, Jashim is left with an OPEN ride with 0 passengers.	He can't go offline ('active ride'), and a private request could land in a ride tied to a stand.	When the last booking leaves an OPEN ride, the ride is cancelled in the same transaction. Private accepts always start a fresh ride.	P6
3	A request that went back to waiting (driver cancelled) expires too early, because expiry counts from created_at.	Nusrat is punished for Jashim's decision.	New column queued_at, reset every time a booking returns to waiting. Expiry counts from it.	P5
4	Expiry is lazy, so a 20-minute-old request can still say REQUESTED, and Jashim's stale screen accepts it.	A ghost booking for someone who gave up.	Accept also requires queued_at within 15 minutes. One expiry helper runs on every read: passenger view, driver list, accept.	P5, P6
5	A pin at Banani's edge whose nearest stand is in Gulshan 1: which zone is the pickup? Snapping can also make pickup zone = drop zone.	Share and private would price the same trip differently; confusing rejections.	Shared: pickup zone = the stand's zone. Private: the pin's zone. Same zone → clear message 'too short for a shared Tesla, try private or walk'.	P4
6	Women-only with multi-seat bookings: companions' gender is unknown.	—	**Accepted.** Benefit of the doubt, documented as assumption A6b.	—
7	If gender can be edited, anyone can switch to join a women-only ride.	Breaks the safety feature.	Gender is set at signup and never editable. Simulated NID verification with a unique NID hash stops second accounts.	P3
8	Nusrat sees Tk 96.25, gets in, Rafiq no-shows, and she's charged Tk 115.00.	Feels like a scam.	Price cap: fare_cap_poysha. **Your price never goes up after you're matched** (Section 7).	P6, P7
9	The assignment brief says 'complete trip', but our ride completes automatically after the last drop.	An reviewer's checklist might not find it.	The last drop button reads 'Drop Shirin & complete trip'.	P12

B. Places where someone gets stuck forever

#	What breaks	Why it matters	How we fix it	Phase
10	Jashim disappears after arriving. Nusrat can't cancel after arrival, and 'one active booking' blocks her from rebooking.	A passenger trapped by someone else's behaviour.	She can cancel if the ride hasn't started 15 minutes after arrival (reason: 'driver didn't start').	P7
11	Jashim disappears mid-trip. Nusrat stays IN_PROGRESS forever.	Same trap.	Lazy cleanup: a ride STARTED over 3 hours ago is auto-completed on read, with an 'auto-closed' event.	P7
12	'1 Tesla online near Banani' counts Jashim when he's busy or closed his laptop an hour ago.	Passengers walk to a stand for nothing.	Count only online drivers with no ride past OPEN and a last_seen_at in the last few minutes (updated on any request or SSE heartbeat).	P4, P8

C. Races we hadn't covered

#	What breaks	Why it matters	How we fix it	Phase
13	Accept locks ride then booking; passenger-cancel locks booking then ride. Opposite order → deadlock.	Postgres kills one transaction; the user sees a 500.	Every transaction locks the ride row first, then bookings. Retry once on deadlock code 40P01.	P6
14	Two tabs request at once; the 'one active booking' index rejects one.	Without handling, that's a crash message instead of a clear answer.	The error handler maps Postgres codes: 23505 (unique) and 23514 (check) → 409 with a readable message.	P0, P5
15	20 parallel OTP guesses all read '0 attempts' before any increment.	The 5-attempt limit is bypassed (brute force).	One atomic step: UPDATE ... SET attempts = attempts + 1 WHERE attempts < 5 RETURNING, then compare.	P3
16	notify() called inside the transaction.	Phones re-fetch before the data is saved and show the old status.	Only notify after the transaction has committed.	P8
17	With a database pool of 1 connection, the Seat Race's two accepts run one after another.	The demo passes without proving anything.	Pool size ≥ 2, each accept in its own transaction, and log both start times in the demo output.	P13

D. Privacy leaks

#	What breaks	Why it matters	How we fix it	Phase
18	The ride timeline shows every event, including 'Rafiq matched, fare 7750'.	Breaks the assignment's rule that passengers see only their own fare.	A passenger's timeline shows only her own booking's events plus ride-level ones (arrived, started). No fares in shared event details.	P7
36	A private request's exact door pin is shown to every online driver before anyone accepts.	Strangers learn where Nusrat lives.	Before accept: 'near Banani Road 11' only. The exact pin is revealed only to the driver who accepts.	P6
37	Logging request bodies writes passwords, OTP codes and NID numbers into the logs.	Secrets sitting in plain text.	Never log bodies by default; redact password, code and nid from anything logged.	P0
40	'No account with this phone' vs 'wrong password'.	Tells an attacker which phones are registered.	One message: 'phone or password is wrong'.	P2

E. Auth and API gotchas

#	What breaks	Why it matters	How we fix it	Phase
19	The JWT was issued before OTP verification, so a check that reads the token still says 'unverified'.	403 right after verifying.	Read verification status and 'has a vehicle' from the database, never from the token.	P2, P3
20	Same idempotency key sent again with a different destination.	Silently returns the old booking.	Store a hash of the request body with the key. Same key + different body → 422.	P5
21	'01700000001' and '+8801700000001' are different strings.	Two accounts for one phone.	A Zod transform normalises every phone number to one format before anything else.	P2
22	Behind Vercel and Render proxies, many users can share one IP.	One person's wrong passwords lock everyone out.	Rate-limit login and OTP by phone number (IP as a second limit). Configure Express 'trust proxy'.	P2
23	Secure cookies can fail on http://localhost in some browsers.	The reviewer can't log in to the Docker version.	COOKIE_SECURE env variable: false locally, true in production.	P2
35	The 7-day JWT expires while Jashim is driving.	He can't drop anyone off.	Sliding session: every request renews the cookie.	P2
38	JWT_SECRET falls back to 'changeme' from .env.example in production.	Anyone can forge a login as Jashim.	Validate env variables with Zod at startup; refuse to start in production if JWT_SECRET is missing or under 32 characters.	P0

F. Lifecycle and driver-side holes

#	What breaks	Why it matters	How we fix it	Phase
30	Private at 1.5 × fare (Tk 172.50) is cheaper than 2 shared seats alone (Tk 230), and comes to the door.	Nobody would ever book multiple shared seats.	Private = seat fare × the Tesla's seats (Nusrat in Bullet: Tk 345.00). Never cheaper than any shared combination.	P4
31	Jashim taps Start 1 minute after arriving, while Shirin is still walking to the stand.	Her history says she rode; she's blocked from rebooking.	A 'boarded' tick per passenger. Start moves boarded riders to IN_PROGRESS; anyone not boarded goes back to the waiting list ('driver left without you').	P7
32	Rafiq cancelled, but his booking still points at Bullet.	Rules could block Shirin's women-only request or measure R2 against Rafiq's old destination.	One activeBookings(rideId) helper used by every rule, seat count, co-rider list and drop list.	P6
33	Accepting after going offline; accepting an Uttara request while in Banani; changing area or seats mid-ride.	Rules bypassed from a stale screen or a direct API call.	Accept checks 'online' inside the transaction; the first accept must match his declared area; area and vehicle edits are blocked during an active ride.	P2, P6
34	POST /driver/ride/requests /:id/drop with another driver's passenger's ID.	Jashim could end someone else's trip.	Every per-booking driver action checks the booking belongs to **his current ride**; otherwise 404.	P7
39	A pin in Chattogram or at (0, 0) snaps to the 'nearest zone'.	Nonsense trips.	Reject pins more than ~3 km from every stand and zone centre: 'outside our service area'.	P4

G. Realtime

#	What breaks	Why it matters	How we fix it	Phase
41	Closed browser tabs stay in the connection map.	Memory grows; notify() writes to dead connections.	Keep a set of connections per user; remove on the request's close event.	P8
42	Some changes nudge nobody: a passenger cancelling, lazy expiry, 'left behind', auto-close.	Stale screens.	A small action → who-gets-nudged table in the code, so nothing is forgotten.	P8

H. Seed, demo, build and deploy

#	What breaks	Why it matters	How we fix it	Phase
24	Scenarios A and B both need Bullet with an active ride; the database's own rules forbid that.	The seed would crash.	Seed only the base world (people, Teslas, stands, a few completed rides for history). Scenarios are scripts, run by tests and by POST /dev/scenario/:name.	P1, P13
25	Vitest runs test files in parallel, all using Nusrat.	Random failures.	fileParallelism: false; reset tables before each file.	P0
26	With DEMO_MODE on in production, anyone can reset the demo.	The demo breaks mid-evaluation.	/dev/* requires a DEMO_KEY header.	P13
27	Neon's pooled connection string and cold starts.	Prepared-statement errors; Render + Neon waking up can time out health checks.	Use Neon's direct connection string; generous health-check timeouts; test a full cold start on Day 4.	P14
28	Server-rendered Next.js fetches don't carry the login cookie.	Random 401s.	Fetch all user data on the client with TanStack Query.	P9
29	Render runs in UTC.	History says 02:41 for an 8:41 AM ride.	Store timestamptz; always format in Asia/Dhaka on the frontend.	P12
43	Next.js reads the rewrite's API_URL during 'next build', not at start-up.	In Docker every API call fails if it wasn't set while building.	Pass API_URL as a Docker build argument; set it in Vercel's build environment.	P9, P14
44	GitHub's default 'Squash and merge'.	Turns six careful, incremental commits into one, destroying the traceable history this document argues for.	Merge with --no-ff, never squash. Cut release/v1.0.0 from pre-release.	All
45	Search for '%' matches everything; removing the map attribution breaks OpenStreetMap's policy.	Minor issues a careful reviewer would surface.	Escape wildcards in the search query; keep Leaflet's attribution line.	P4, P11

Review Sufficiency

The second review pass surfaced fewer and lower-severity issues than the first — predominantly missing checks rather than structural design flaws — indicating diminishing returns from further review at the design stage. Issues discovered during implementation are recorded and, where they alter a documented rule, reflected back into this document.

20. Implementation Roadmap

Implementation follows dependency order: foundational infrastructure, then Dhaka reference data, then the driver-side domain, then the passenger-side domain, then the pooling and trip logic that depends on both, then real-time delivery. The backend is completed and fully tested before frontend work begins, since correctness risk is concentrated there.

Process for each phase

- Work proceeds on a dedicated `feature/*` branch.
- Implementation proceeds in small, independently working increments, each committed separately.
- Each phase is verified with an automated test (Vitest + Supertest) before being considered complete.
- The branch is merged into master with `git merge --no-ff` once verified, preserving the merge in history.

Backend (Phases 0-8)

Phase	Scope	Branch	Verification	Risk items addressed
0 — Foundation	Docker + Postgres, health check, structured logging with request IDs, centralised error handling, environment validation, test harness.	feature/project-setup	Service starts; health check returns 200; test suite runs.	#14, #25, #37, #38
1 — Reference data	Zones, inter-zone distances, Tesla stands and landmarks, seeded.	feature/dhaka-map-data	All zone-pair distances present; seed is idempotent.	#24
2 — Driver onboarding	Authentication (shared by both roles), vehicle registration, online/offline state.	feature/auth, feature/vehicle-onboarding	Auth and authorisation boundary conditions covered by tests.	#19, #21, #22, #23, #33, #35, #40
3 — Identity verification	Phone OTP and simulated NID verification; gender locked at signup.	feature/identity-verification	Attempt limits and duplicate-NID rejection verified under concurrent load.	#7, #15, #19
4 — Discovery and pricing	Place search, nearest-stand resolution, fare service as pure functions.	feature/places-and-fare	All published fare figures reproduced exactly by tests.	#5, #12, #30, #39, #45
5 — Ride requests	Idempotent request creation, history, cancellation, lazy expiry.	feature/ride-requests	Idempotency and ownership-isolation tests pass.	#3, #4, #14, #20
6 — Pooling (core)	Matching rules R1–R6, the locked accept transaction, private hire, seat invariant.	feature/tesla-pooling	Concurrent-accept test passes consistently across repeated runs.	#1, #2, #4, #8, #13, #32, #33, #36
7 — Trip lifecycle	State machine, boarding, no-show, per-passenger drop-off, stuck-state recovery.	feature/trip-lifecycle	Every transition-table cell covered; price-cap behaviour verified.	#1, #8, #10, #11, #18, #31, #34
8 — Real-time delivery	SSE connection management, commit-gated notifications.	feature/realtime-sse	Notification observed only after transaction commit; closed connections release cleanly.	#12, #16, #41, #42

Frontend and Release (Phases 9-14)

Phase	Scope	Branch
9 — Frontend foundation	Next.js scaffold, API proxy, data-fetching provider, authentication screens.	feature/web-setup
10 — Driver screens	Vehicle onboarding form, online toggle, request list.	feature/driver-ui
11 — Passenger screens	Map, search, fare estimate, request flow.	feature/passenger-ui, feature/map-search
12 — Ride screens	Full driver and passenger ride views with loading/empty/error states for all five scenarios.	feature/ride-ui
13 — Concurrency demonstration	Seat Race tool and scenario loader, gated behind a demo flag.	feature/seat-race-demo
14 — Release	Deployment, documentation, tagged release build.	pre-release, release/v1.0.0

Contingency

If schedule pressure requires scope reduction beyond Section 15's plan, the ride-replay timeline UI and the no-show countdown display are the next candidates to defer; the underlying backend logic for both is unaffected.

Appendix: Diagram Source (Mermaid)

The following Mermaid definitions correspond to the diagrams in this document and render automatically in GitHub-flavoured Markdown, for direct reuse in the project README.

```
flowchart LR
  B[Browser<br/>Nusrat / Jashim] -->|pages| N[Next.js on Vercel]
  N -->|/api/* rewrite| A[Express API on Render]
  A -->|SQL in transactions| D[(PostgreSQL on Neon)]
  A -. SSE nudge .-> B
```

```
erDiagram
  USERS ||--o| VEHICLES : drives
  USERS ||--o{ RIDE_REQUESTS : books
  VEHICLES ||--o{ RIDES : makes
  RIDES ||--o{ RIDE_REQUESTS : "pool members"
  RIDES ||--o{ RIDE_EVENTS : logs
  RIDE_REQUESTS ||--o{ RIDE_EVENTS : logs
  ZONES ||--o{ PLACES : contains
  ZONES ||--o{ ZONE_DISTANCES : from
  ZONES ||--o{ RIDE_REQUESTS : "pickup / drop"
  RIDES {
    uuid id PK
    uuid vehicle_id FK
    text status
    smallint capacity
    smallint seats_taken "CHECK <= capacity"
    boolean is_private
  }
  RIDE_REQUESTS {
    uuid id PK
    uuid passenger_id FK
    uuid ride_id FK "null while waiting"
    text ride_type "SHARED or PRIVATE"
    boolean women_only
    int total_fare_poysha
  }
  OTP_CODES {
    uuid id PK
    text phone
    text code_hash
    timestamptz expires_at
  }
```

```
stateDiagram-v2
  [*] --> REQUESTED
  REQUESTED --> MATCHED : driver accepts
  REQUESTED --> CANCELLED : passenger
  REQUESTED --> EXPIRED : 15 min
  MATCHED --> DRIVER_ARRIVED : driver arrived
  MATCHED --> CANCELLED : passenger
  MATCHED --> REQUESTED : driver cancels ride
  DRIVER_ARRIVED --> IN_PROGRESS : start (fare locked)
  DRIVER_ARRIVED --> REQUESTED : driver cancels ride
  DRIVER_ARRIVED --> NO_SHOW : after 5 min wait
  IN_PROGRESS --> COMPLETED : dropped off
```

```
flowchart TB
  Apps[Passenger + driver apps] --> LB[Load balancer + rate limits]
  LB --> API[API servers x N]
  LB --> RT[Realtime SSE servers x N]
  LB --> LOC[Location service]
  API --> PG[(Postgres primary)]
  API --> RR[(Read replicas)]
  API --> Q[Queue + workers]
  API --> R[(Redis pub/sub + geo)]
  R --> RT
  LOC --> R
```